

Transformer-Encoder



MLP Lab.

Contents

01 Introduction of Transformer

1) Encoder-Decoder Structure

2) Transformer Structure

2) Usage of Encoder-Decoder

02 Transformer Encoder

1) Self-Attention

2) Multi-Head Attention

3) Positional Encoding

4) Add & Norm



Contents

01 Introduction of Transformer

- 1) Encoder-Decoder Structure

- 2) Transformer Structure

- 2) Usage of Encoder-Decoder

02 Transformer Encoder

- 1) Self-Attention

- 2) Multi-Head Attention

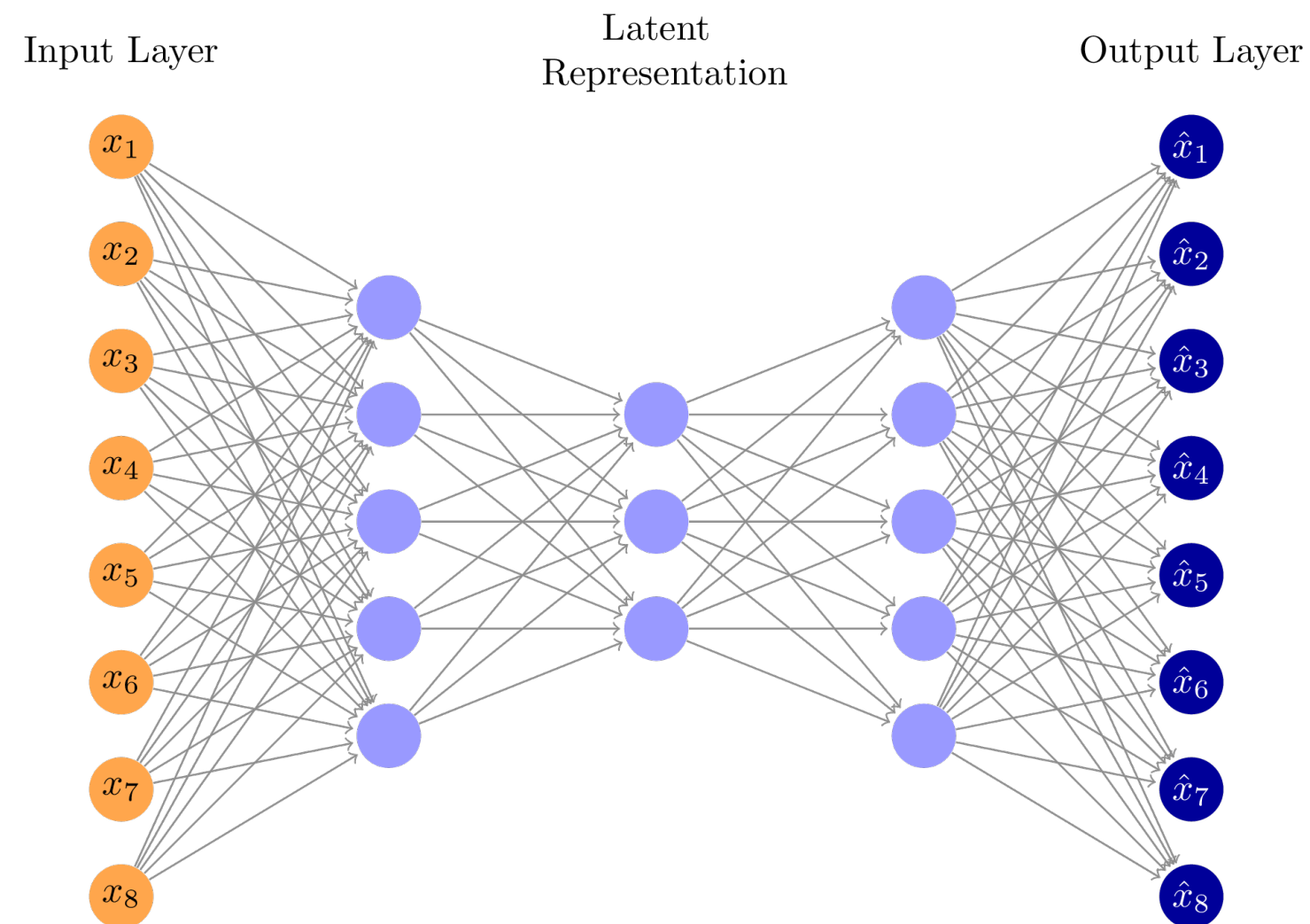
- 3) Positional Encoding

- 4) Add & Norm



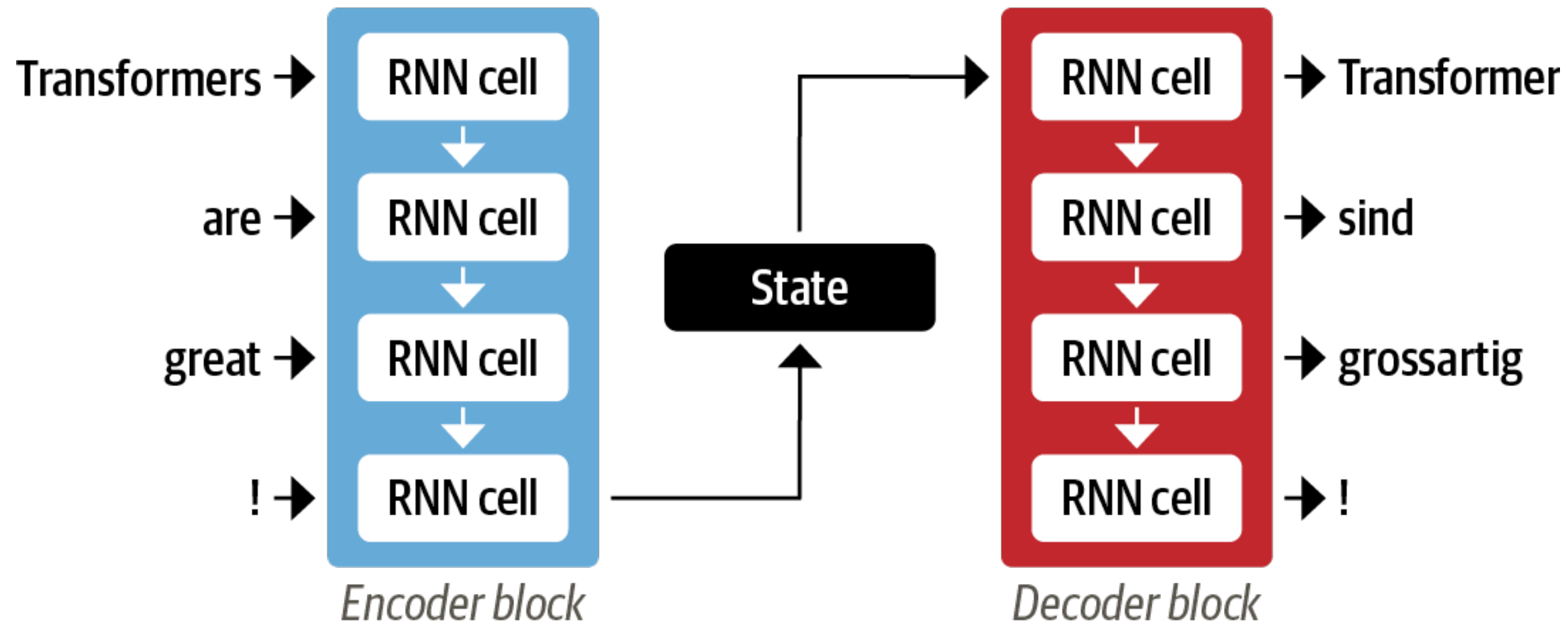
01 Encoder-Decoder Structure

- Auto Encoder : Leverage Bottleneck to put Representations into Latent Space
 - It compresses the information in the input data and then restores it, looking for the most important features.



01 Encoder-Decoder Structure

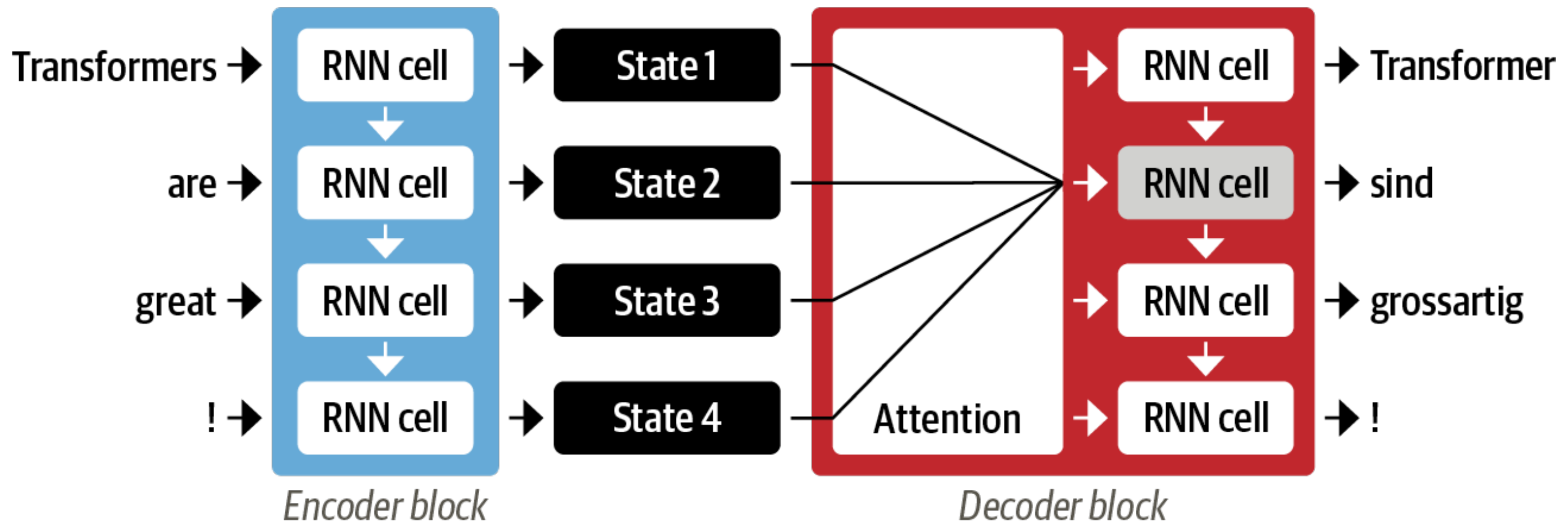
- Encoder-Decoder on RNN: Using RNNs to represent text in latent space



✓ The information bottleneck has occurred because of the use of the last hidden state.

01 Encoder-Decoder Structure

- Encoder-Decoder on RNN+Attention: Using RNN+att to represent text in latent space

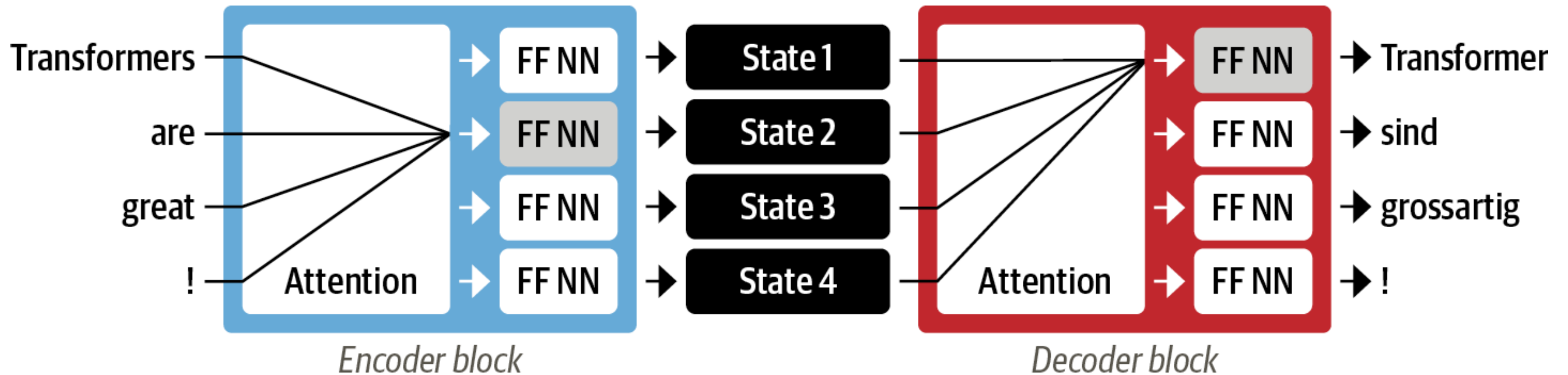


✓ Resolve information bottlenecks

✓ Non-parallelize across input sequences

01 Encoder-Decoder Structure

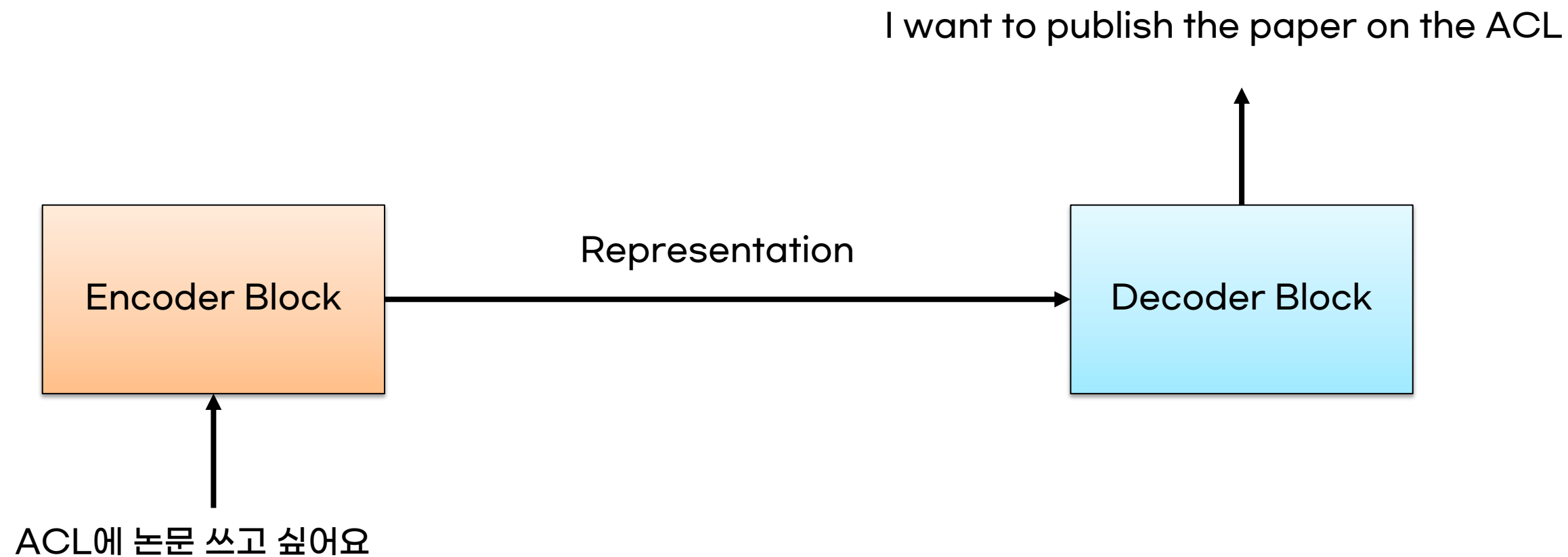
- Enc-Dec on Transformer: Only using the attention to represent text in latent space.



- ✓ Resolve information bottlenecks
- ✓ Parallelize across input sequences

01 Encoder-Decoder Structure

- Transformer Encoder-Decoder Architecture: Without Bottleneck
 - ✓ The size of input and output embedding are the same size
 - ✓ Since it encodes via self-attention, it does not seem to require a bottleneck.



02 Transformer Structure

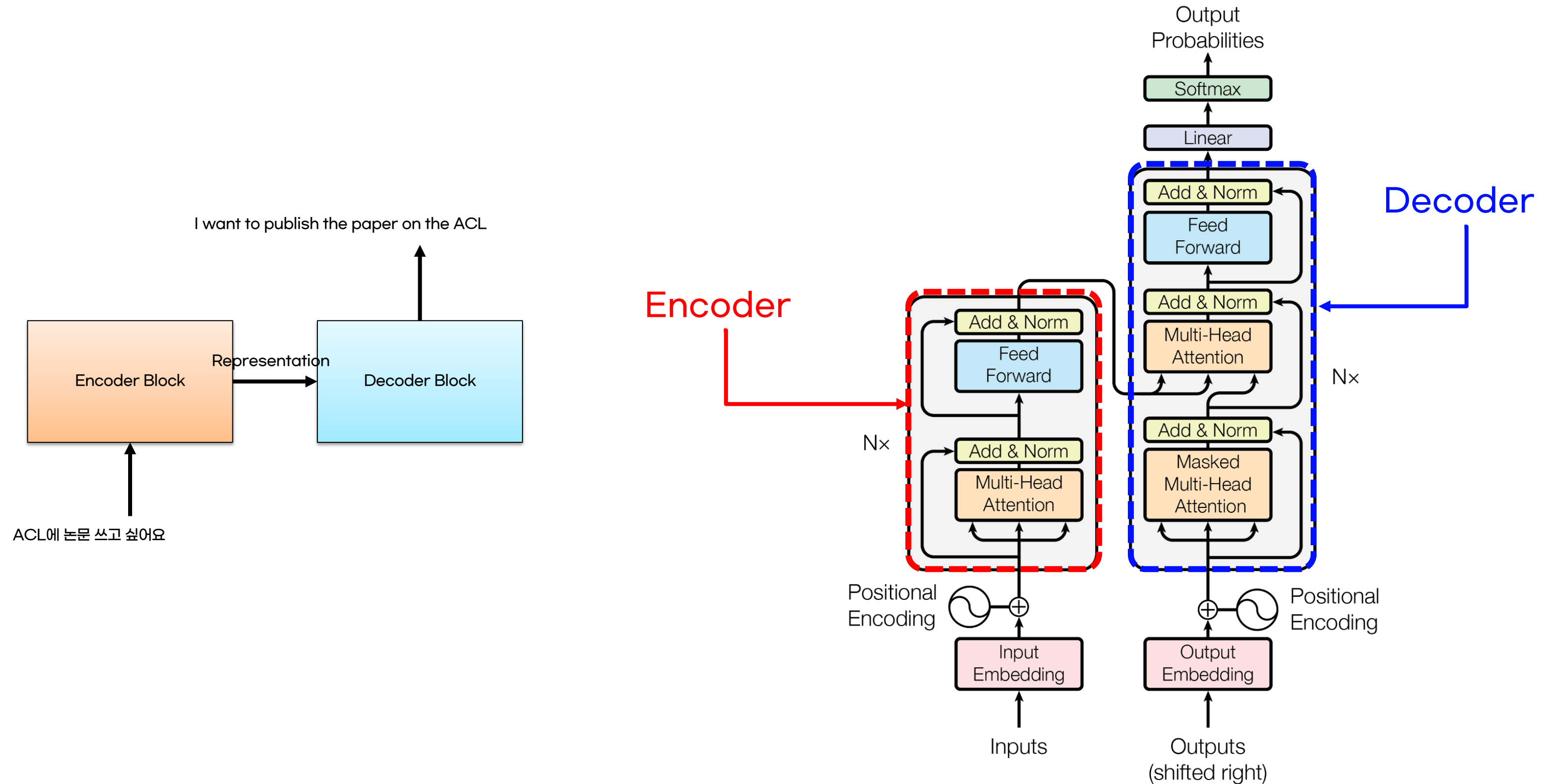
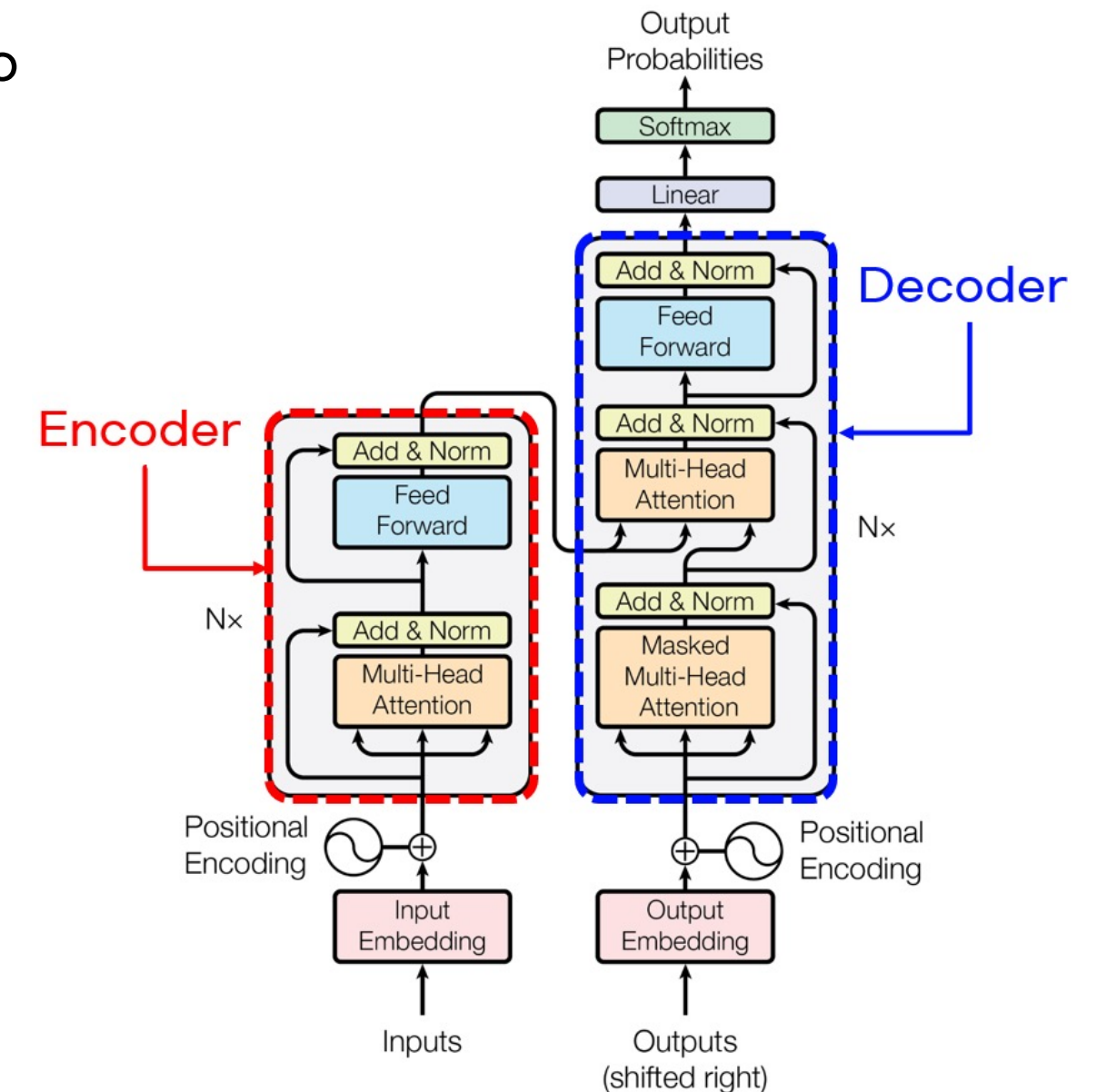


Figure 1: The Transformer - model architecture.

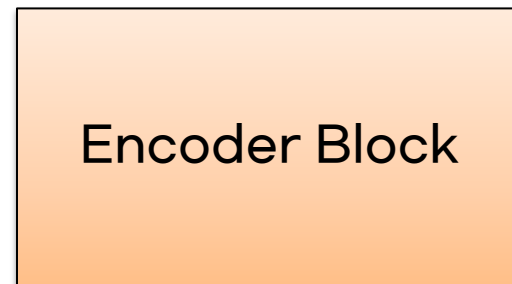
02 Transformer Structure

- Tokenize the input sentence through a tokenizer, convert it to a token embedding, and combine the position embedding and token embedding to inject information about the position of the token into the encoder.
- Encoders and decoders consist of a stack of encoder layers and decoder layers, similar to the stack of convolutional layers in computer vision.
- The output of the encoder is injected into each decoder layer, and the decoder predicts the next most likely token in the sequence, predicts the <EOS> token, or repeats until the maximum length is reached.
- The transformer architecture is designed for seq2seq operations, but the encoder and decoder blocks are independent models.

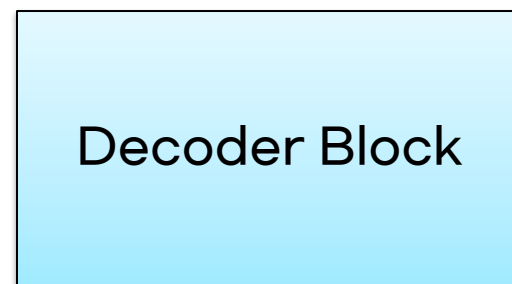


03 Usage of Encoder-Decoder

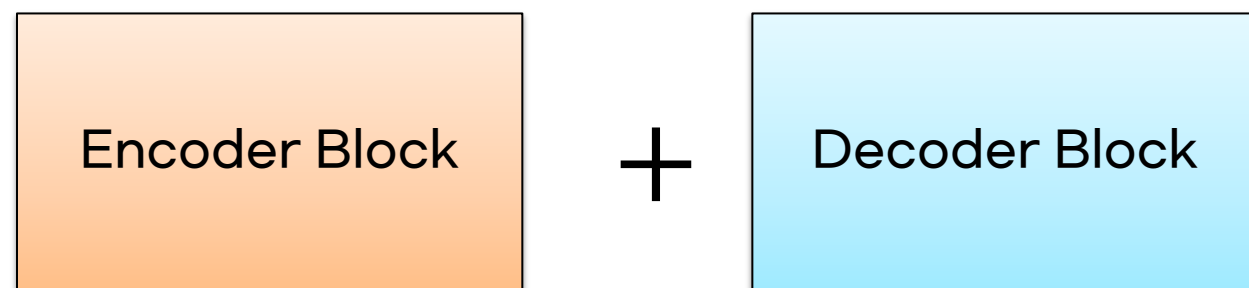
- Transformer Encoder-Decoder Architecture.



- Encoder block : BERT Series Models for QA, text classification



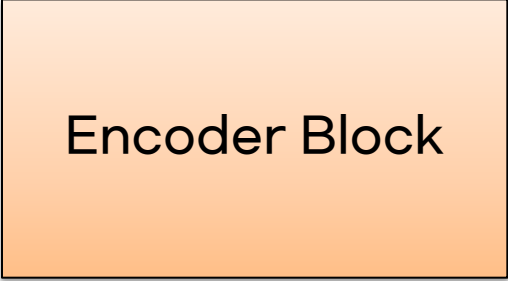
- Decoder block : GPT-like models for Chatbot, NLG, QA



- Encoder block + Decoder block : BART, T5 for Machine Translation and Summarization

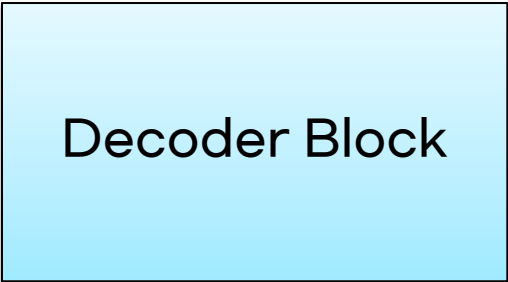
03 Usage of Encoder-Decoder

- Transformer Encoder-Decoder Architecture.



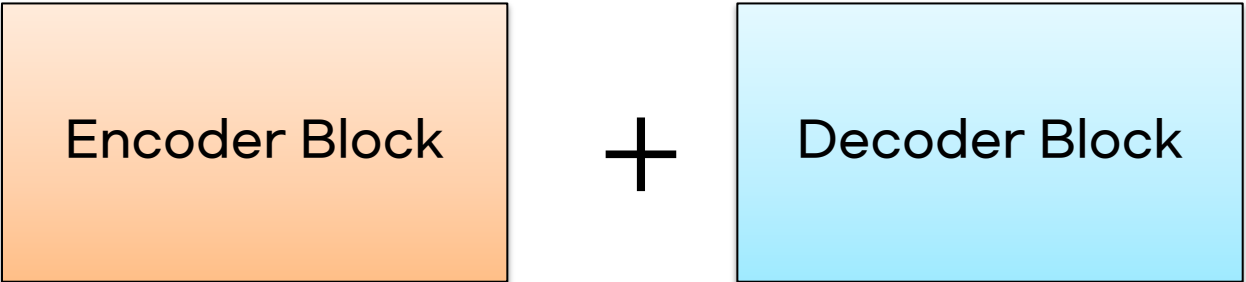
Encoder Block

- Encoder block : BERT Series Models for QA, text classification



Decoder Block

- Decoder block : GPT-like models for Chatbot, NLG, QA



Encoder Block

Decoder Block

- Encoder block + Decoder block : BART, T5 for Machine Translation and Summarization

Contents

01 Introduction of Transformer

1) Encoder-Decoder Structure

2) Transformer Structure

2) Usage of Encoder-Decoder

02 Transformer Encoder

1) Self-Attention

2) Multi-Head Attention

3) Positional Encoding

4) Add & Norm

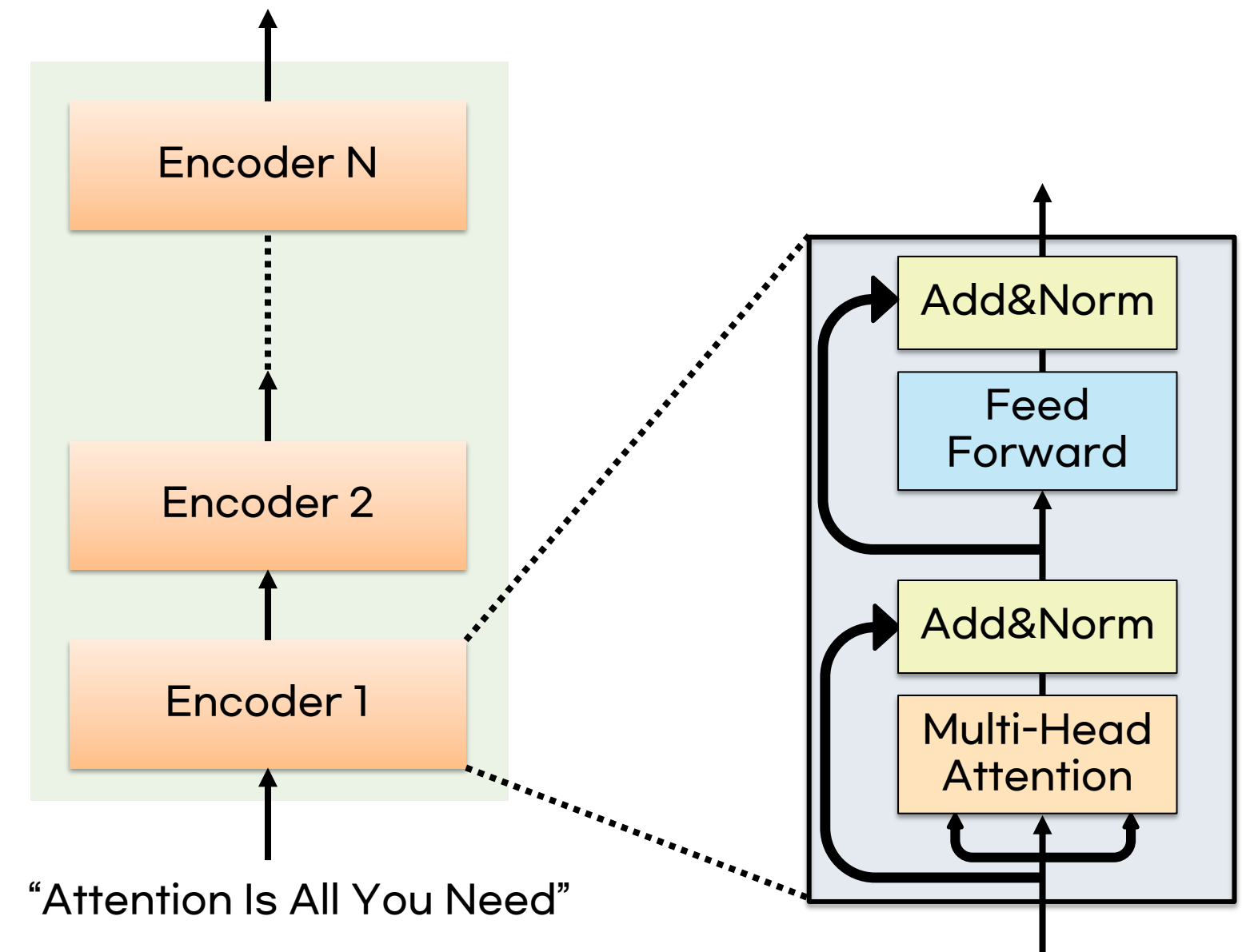


00 Introduction of Transformer Encoder

- Use the number of N encoders as a stacked structure
- The first encoder (Encoder 1) receives the sentence as input
- The next encoder takes the output of the previous encoder

- Transformer Encoder Components

- ✓ Multi-Head Attention
- ✓ Feed-Forward
- ✓ Skip-Connection
- ✓ Add & Norm

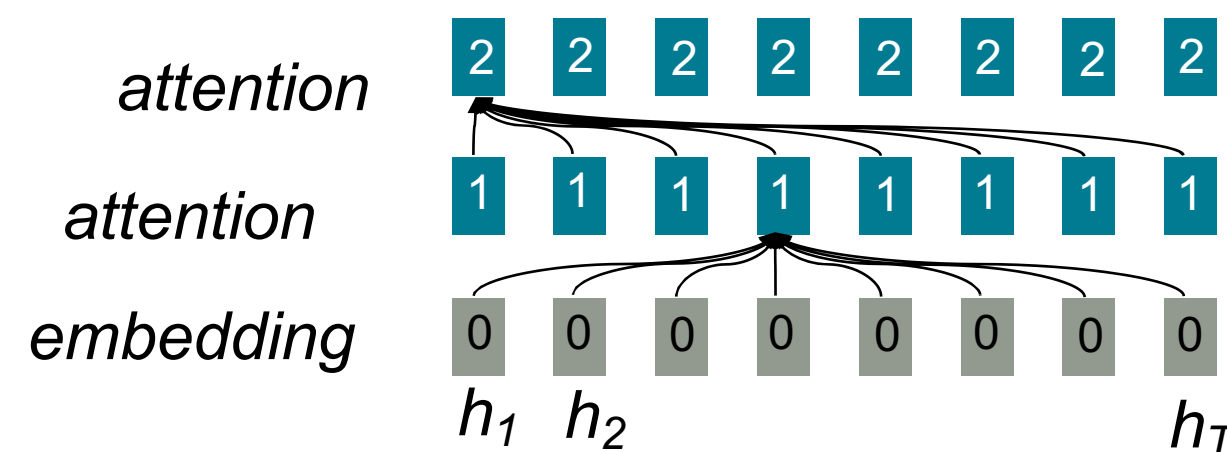


"Attention Is All You Need"

Transformer Encoder
Architecture

01 Attention

- **Attention** treats each word's representation as a **query** to access and incorporate information from **a set of values**.
- Number of unparallelizable operations does not increase with sequence length.
- Maximum interaction distance: $O(1)$, since all words interact at every layer!

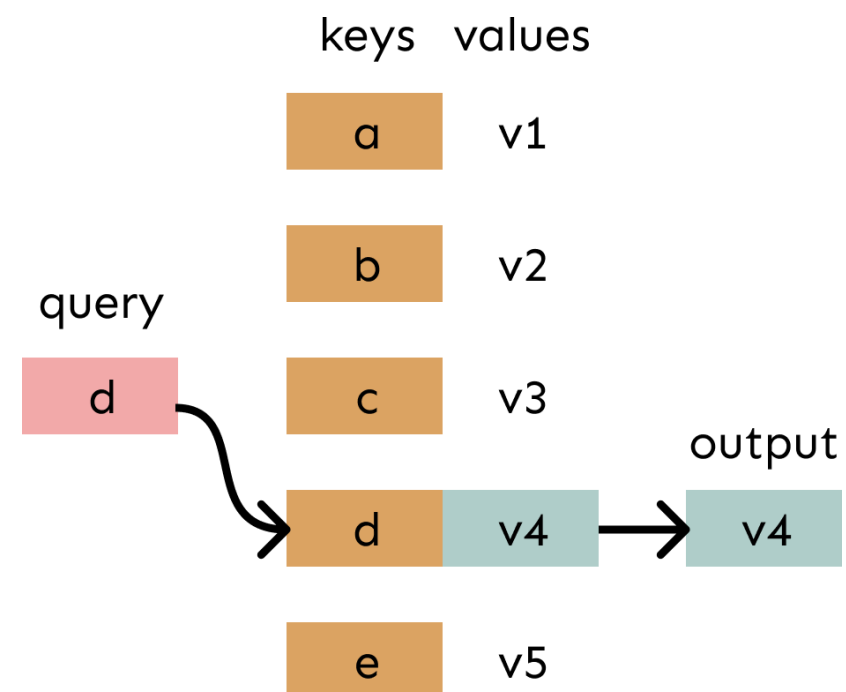


All words attend to all words in previous layer; most arrows here are omitted

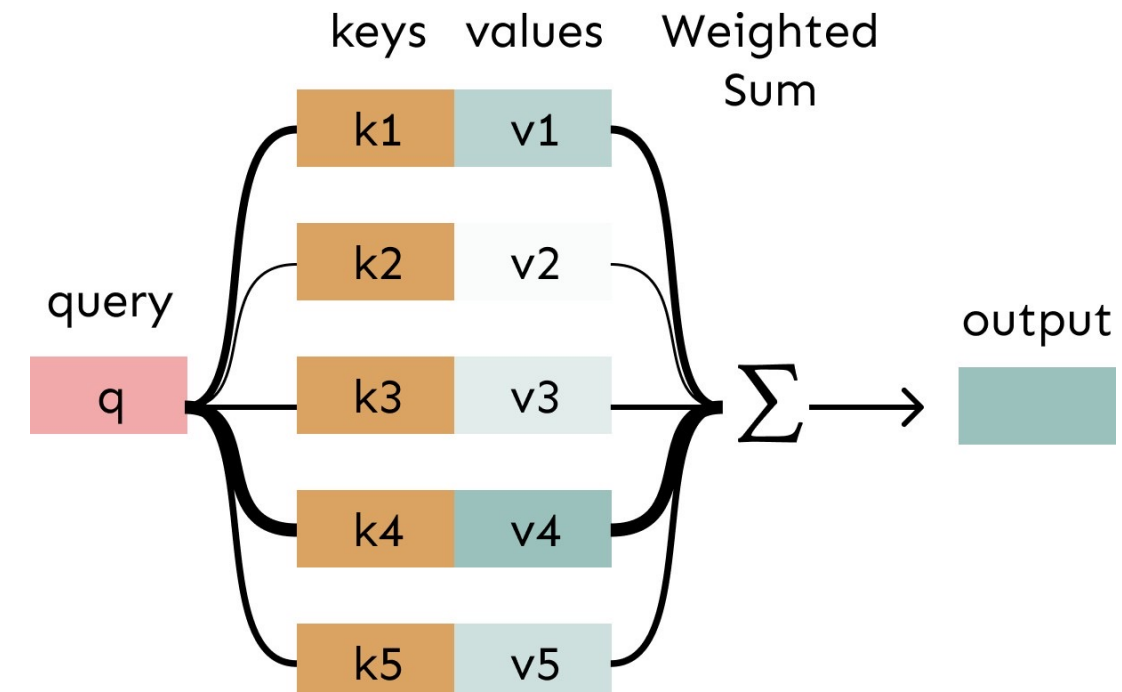
01 Attention

- We can think of **attention** as performing fuzzy lookup in a key-value store

In a **lookup table**, we have a table of **keys** that map to **values**. The **query** matches one of the keys, returning its value.



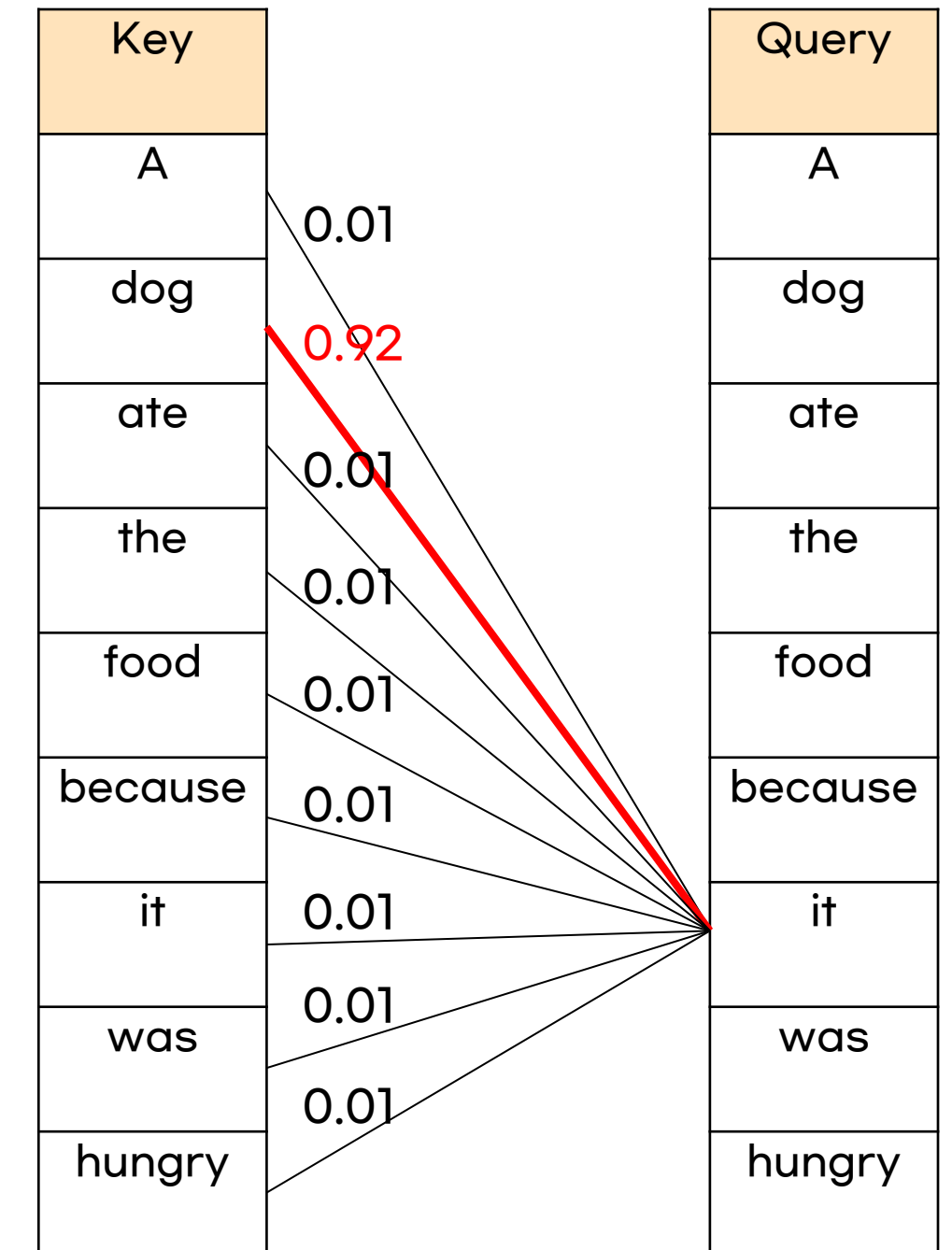
In **attention**, the **query** matches all **keys** *softly*, to a weight between 0 and 1. The keys' **values** are multiplied by the weights and summed.



01 Self-Attention

Introduction of Self-Attention

- Methods for calculating the association (similarity) between a specific word and all words in **a sentence**
- Select a specific word as query to help the system understand the context by calculating the association with all the words in a sentence



01 Self-Attention

1. Let $\mathbf{w}_{1:n}$ be a sequence of words in vocabulary V , like *Zuko made his uncle tea*.
2. For each $\mathbf{w}_i$, let $\mathbf{x}_i = E\mathbf{w}_i$, where $E \in \mathbb{R}^{d \times |V|}$ is an embedding matrix.
3. Transform each word embedding with weight matrices Q, K, V , each in $\mathbb{R}^{d \times d}$

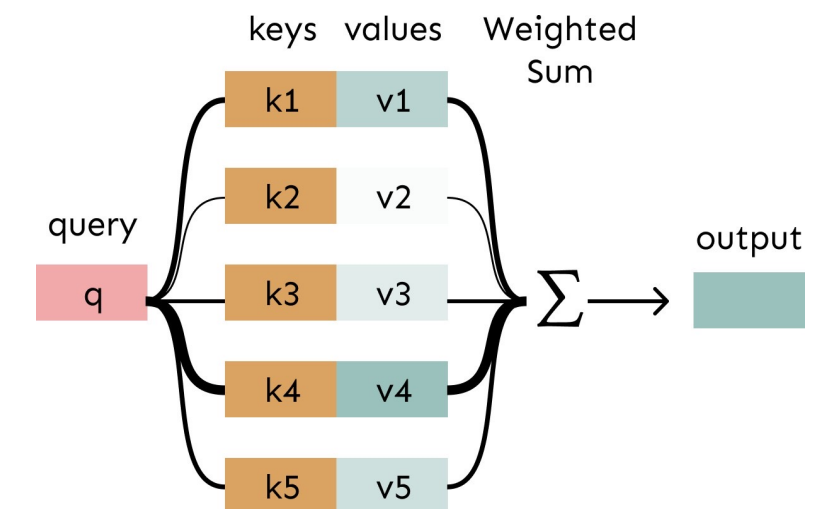
$$\mathbf{q}_i = Q\mathbf{x}_i \text{ (queries)} \quad \mathbf{k}_i = K\mathbf{x}_i \text{ (keys)} \quad \mathbf{v}_i = V\mathbf{x}_i \text{ (values)}$$

4. Compute pairwise similarities between keys and queries; normalize with softmax

$$\mathbf{e}_{ij} = \mathbf{q}_i^\top \mathbf{k}_j \quad \alpha_{ij} = \frac{\exp(\mathbf{e}_{ij})}{\sum_{j'} \exp(\mathbf{e}_{ij'})}$$

5. Compute output for each word as weighted sum of values

$$\mathbf{o}_i = \sum_j \alpha_{ij} \mathbf{v}_j$$



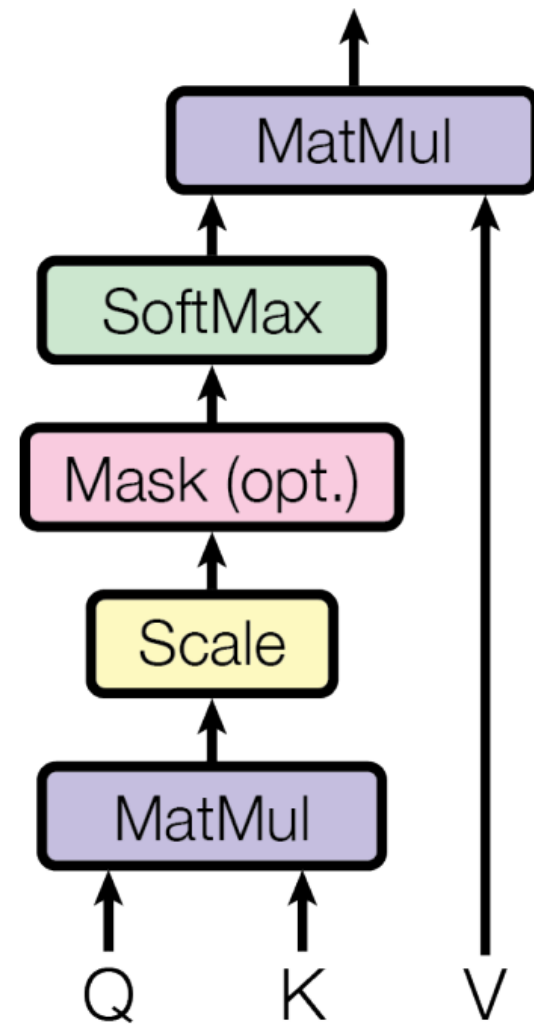
01 Self-Attention

Self Attention – Process Summary

1. Extract embeddings from a sentence: create an embeddings matrix
2. Create Key, Query, Value from Embedded Matrix
3. Calculate the similarity between Query and Key : $Q \cdot K^T$
4. Scaling and Softmax (normalization) $\rightarrow$ score matrix : $\text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right)$
5. Multiply by Value to obtain Attachment matrix Z

01 Self-Attention (a toy example)

Scaled Dot-Product Attention



$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{Q K^T}{\sqrt{d_k}} \right) V$$

$$Q = X \times W_Q$$

$$K = X \times W_K$$

$$V = X \times W_V$$

$X = \text{input embedding}$

01 Self-Attention (a toy example)

1. Example of Query, Key, Value generation

The dimensionality of a word: 4
 Number of words : 3

$$\begin{array}{c}
 I \\
 am \\
 good
 \end{array}
 \overset{X}{\begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 2 & 0 & 2 \\ 1 & 1 & 1 & 1 \end{bmatrix}}
 \times
 \overset{W_Q}{\begin{bmatrix} 1 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix}}
 =
 \overset{Q}{\begin{bmatrix} 1 & 0 & 2 \\ 2 & 2 & 2 \\ 2 & 1 & 3 \end{bmatrix}}$$

01 Self-Attention (a toy example)

1. Example of Query, Key, Value generation

The dimensionality of a word: 4
Number of words : 3

Randomly initialized and then
update at training time

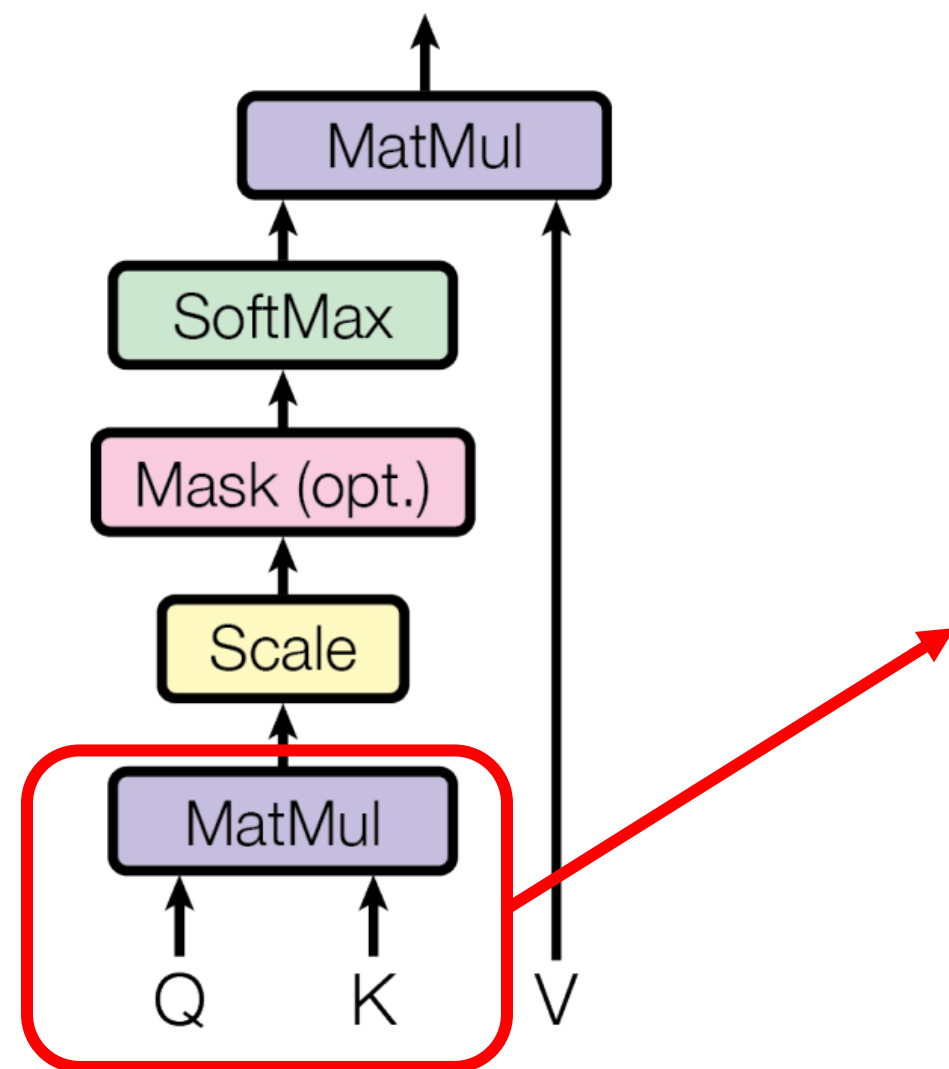
$$\begin{array}{c} I \\ am \\ good \end{array} \overset{X}{\begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 2 & 0 & 2 \\ 1 & 1 & 1 & 1 \end{bmatrix}} \times \overset{W_Q}{\begin{bmatrix} 1 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix}} = \overset{Q}{\begin{bmatrix} 1 & 0 & 2 \\ 2 & 2 & 2 \\ 2 & 1 & 3 \end{bmatrix}}$$

$$X \times W_K = K$$

$$X \times W_V = V$$

01 Self-Attention (a toy example)

Scaled Dot-Product Attention

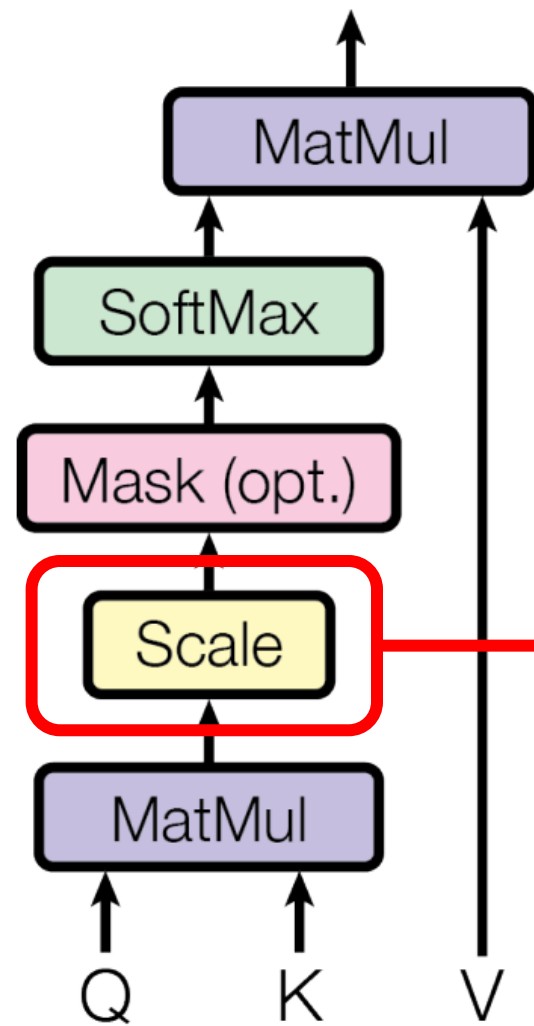


$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{Q K^T}{\sqrt{d_k}} \right) V$$

$$QK^T = \begin{bmatrix} 1 & 0 & 2 \\ 2 & 2 & 2 \\ 2 & 1 & 3 \end{bmatrix} \times \begin{bmatrix} 0 & 4 & 2 \\ 1 & 4 & 3 \\ 1 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 2 & 4 & 4 \\ 4 & 16 & 12 \\ 4 & 12 & 10 \end{bmatrix}$$

01 Self-Attention (a toy example)

Scaled Dot-Product Attention

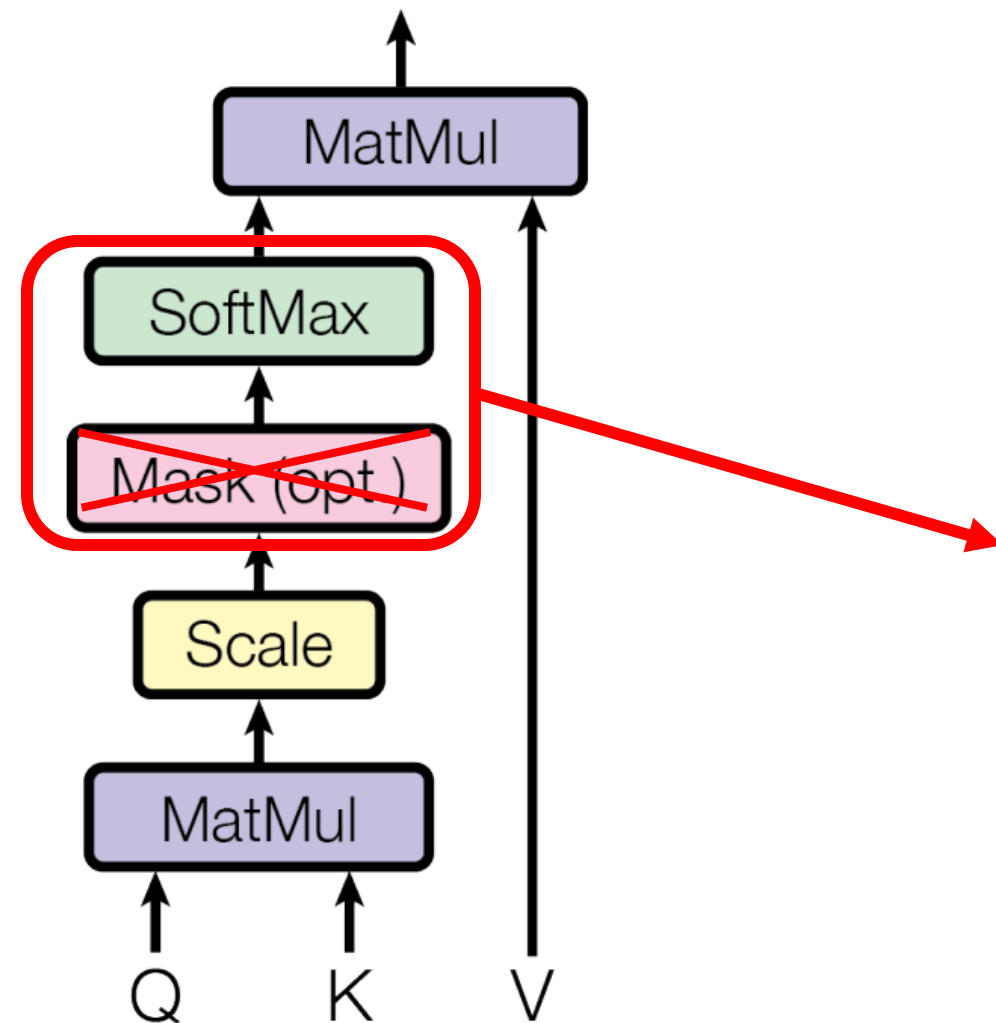


$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{Q K^T}{\sqrt{d_k}} \right) V$$

$$\frac{Q K^T}{\sqrt{d_k}} = \frac{1}{\sqrt{3}} \begin{bmatrix} 2 & 4 & 4 \\ 4 & 16 & 12 \\ 4 & 12 & 10 \end{bmatrix} = \begin{bmatrix} 1.15 & 2.31 & 2.31 \\ 2.31 & 9.24 & 6.93 \\ 2.31 & 6.93 & 5.77 \end{bmatrix}$$

01 Self-Attention (a toy example)

Scaled Dot-Product Attention

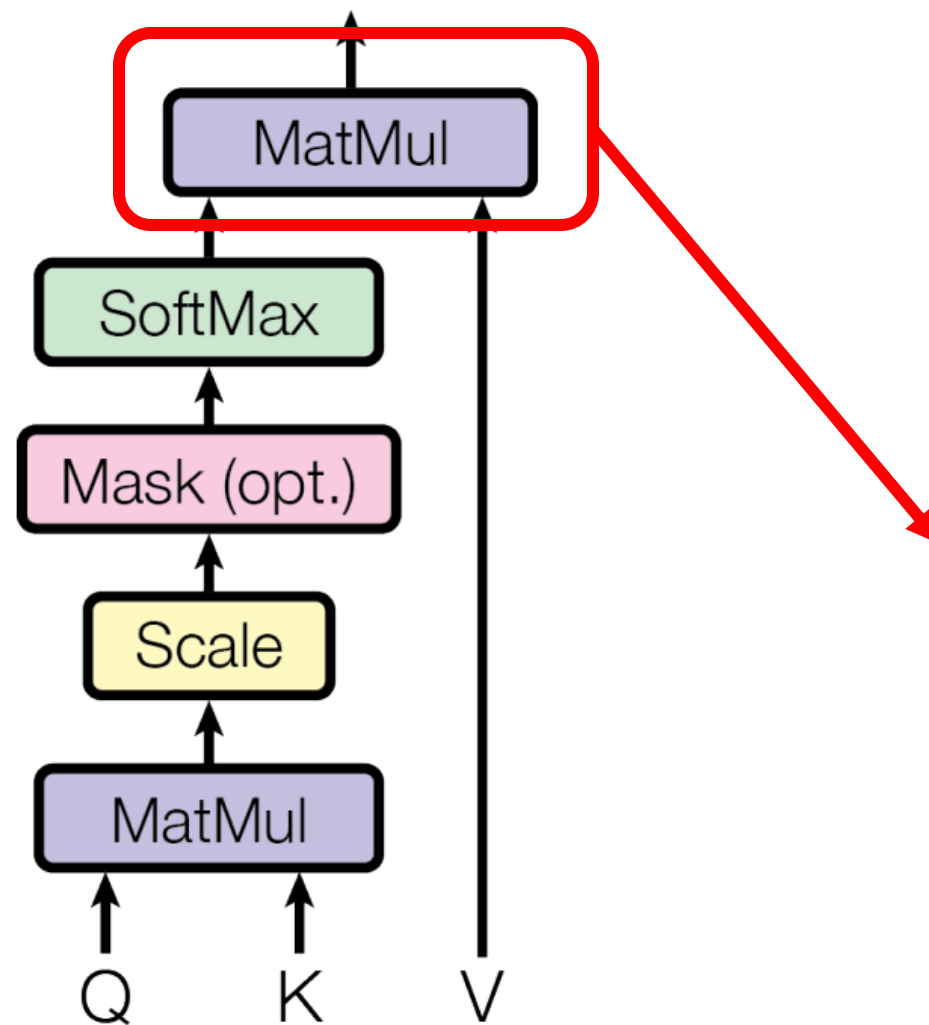


$$Attention(Q, K, V) = \text{softmax} \left(\frac{Q K^T}{\sqrt{d_k}} \right) V$$

$$\text{softmax} \left(\frac{Q K^T}{\sqrt{d_k}} \right) = \begin{bmatrix} 0.14 & 0.43 & 0.43 \\ 0.00 & 0.91 & 0.09 \\ 0.01 & 0.75 & 0.24 \end{bmatrix}$$

01 Self-Attention (a toy example)

Scaled Dot-Product Attention



$$Attention(Q, K, V) = \text{softmax} \left(\frac{Q K^T}{\sqrt{d_k}} \right) V$$

$$\text{softmax} \left(\frac{Q K^T}{\sqrt{d_k}} \right) V$$

$$= \begin{bmatrix} 0.14 & 0.43 & 0.43 \\ 0.00 & 0.91 & 0.09 \\ 0.01 & 0.75 & 0.24 \end{bmatrix} \begin{bmatrix} 1 & 2 & 3 \\ 2 & 8 & 0 \\ 2 & 6 & 3 \end{bmatrix}$$

$$= \begin{bmatrix} 1.86 & 6.32 & 1.70 \\ 2.00 & 7.81 & 0.27 \\ 2.00 & 7.48 & 0.74 \end{bmatrix}$$

01 Self-Attention (a toy example)

- ✓ Self-attention consider the relationship between all word pairs in the sequence

Scaled Dot-Product Attention



01 Self-Attention (A real-word example)

Self Attention – Process

1. Extract embeddings from a sentence: create an embeddings matrix
2. Create Key, Query, Value from Embedded Matrix
3. Calculate the similarity between Query and Key : $Q \cdot K^T$
4. Scaling and Softmax (normalization) $\rightarrow$ score matrix : $\text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right)$
5. Multiply by Value to obtain Attachment matrix Z

01 Self-Attention (A real-word example)

Terms

d_{emb} : Dimension of Sequence embedding

d_k : Dimension of Query, Key, Value - Query, Key, Value

n_{tok} : Number of Tokens

$\mathbb{M}^{N \times M}$: $N \times M$ matrix

01 Self-Attention (A real-word example)

Self Attention – Process

1. Extract embeddings from a sentence: create an embeddings matrix
2. Create Key, Query, Value from Embedded Matrix
3. Calculate the similarity between Query and Key : $Q \cdot K^T$
4. Scaling and Softmax (normalization) $\rightarrow$ score matrix : $\text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right)$
5. Multiply by Value to obtain Attachment matrix Z

01 Self-Attention (A real-word example)

$$X \in \mathbb{M}^{n_{tok} \times d_{emb}}$$

1. Extract embeddings from a sentence: create an embeddings matrix

Ex) “I am good”, $d_{emb} = 768$

x_1 : I, x_2 : am, x_3 : good

$$x_1 = [2.11, 0.42, 1.17, \dots, 5.32]$$

$$x_2 = [1.41, 4.22, 3.12, \dots, 2.77]$$

$$x_3 = [4.16, 2.48, 0.24, \dots, 4.94]$$

$$X = \text{concat}[x_1, x_2, x_3] = \begin{bmatrix} 2.11 & 0.42 & 1.17 & \dots & 5.32 \\ 1.41 & 4.22 & 3.12 & \dots & 2.77 \\ 4.16 & 2.48 & 0.24 & \dots & 4.94 \end{bmatrix}$$

Size of $X = [\# \text{ tokens} \times d_{emb}]$

01 Self-Attention (A real-word example)

1. Extract embeddings from a sentence: create an embeddings matrix with `(nn.Embedding)`

`nn.Embedding?`

Create a trainable look-up table to get the embedding for each token.

- 1) Generate `vocab2idx`
- 2) Convert each token to an index
- 3) Extract rows from the lookup table corresponding to the index and use them as embedding
- 4) Update the embeddings while Back-propagation

01 Self-Attention (A real-word example)

1. Extract embeddings from sentences and generate embedding matrix X
2. Ex) "I am good", $d_{emb} = 768$

Bert-base-uncased

$n_{tok} = 30522$

$d_{emb} = 768$

→ Look-up-table $\in \mathbb{M}^{30522 \times 768}$

Embedding Extraction Method: Using nn.Embedding (Look-up Table)

```
from torch import nn
from transformers import AutoConfig

config = AutoConfig.from_pretrained(model_ckpt)
token_emb = nn.Embedding(config.vocab_size, config.hidden_size)
token_emb
```

✓ 0.9s

Embedding(30522, 768)

```
for _, param in token_emb.named_parameters():
    print(param.requires_grad)
```

✓ 0.3s

True

01 Self-Attention (A real-word example)

$$\begin{aligned} n_{tok} &= 3 \\ d_{emb} &= 768 \\ \rightarrow X &\in \mathbb{M}^{3 \times 768} \end{aligned}$$

1. Extract embeddings from sentences and generate embedding matrix X

Ex) “I am good”, emb_dim = 768

Embedding Extraction Method: Using nn.Embedding (Look-up Table)

inputs

✓ 0.4s

```
{'input_ids': tensor([[1045, 2572, 2204]]), 'token_type_ids': tensor([[0, 0, 0]]),  
'attention_mask': tensor([[1, 1, 1]])}
```

```
inputs_embeds = token_emb(inputs.input_ids)  
inputs_embeds.size()
```

✓ 0.4s

```
torch.Size([1, 3, 768])
```

inputs_embeds

✓ 0.5s

```
tensor([[[ 0.3340, -0.0996,  2.7319, ...,  0.8605, -0.7828,  0.2881],  
         [-1.4061, -0.8634, -0.0905, ..., -0.7111, -0.1840, -0.7100],  
         [-2.1451, -0.1344,  0.8248, ..., -1.7216,  0.2581,  0.0751]]],  
        grad_fn=<EmbeddingBackward0>)
```

1. When tokenizing a sentence, get each word's index (input_ids) of each word when tokenizing
2. Get the embedding vectors corresponding to each input_ids from token_emb(Look-up Table).
3. Sum the embedding of each token extracted from the look-up table to get the embedding matrix.

01 Self-Attention (A real-word example)

Self Attention – Process

1. Extract embeddings from a sentence: create an embeddings matrix
2. Create Key, Query, Value from Embedded Matrix
3. Calculate the similarity between Query and Key : $Q \cdot K^T$
4. Scaling and Softmax (normalization) $\rightarrow$ score matrix : $\text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right)$
5. Multiply by Value to obtain Attachment matrix Z

01 Self-Attention (A real-word example)

2. Create Key, Query, Value from Embedded Matrix

- For generating Key(K), Query(Q), Value(V) matrixes, set weight parameters W^K, W^Q, W^V
where $W^K, W^Q, W^V \in \mathbb{M}^{d_{emb} \times d_k}$ are **trainable parameters**
- Generate K , Q and V by multiplying the weight matrices W^K, W^Q, W^V and the embedding matrix X you set.

$$K = XW^K$$

$$Q = XW^Q$$

$$V = XW^V$$

where X is sequence embedding matrix

01 Self-Attention (A real-word example)

$$X \in \mathbb{M}^{n_{tok} \times d_{emb}}$$

2. Create Key, Query, Value from Embedded Matrix

$$W^K, W^Q, W^V \in \mathbb{M}^{d_{emb} \times d_k}$$

$$X \in \mathbb{M}^{n_{tok} \times d_{emb}}$$

$$K = XW^K \in \mathbb{M}^{n_{tok} \times d_k}$$

$$Q = XW^Q \in \mathbb{M}^{n_{tok} \times d_k}$$

$$V = XW^V \in \mathbb{M}^{n_{tok} \times d_k}$$

01 Self-Attention (A real-word example)

2. Create Key, Query, Value from Embedded Matrix

- For generating Key(K), Query(Q), Value(V) matrixes, set weight parameters W^K, W^Q, W^V

where $W^K, W^Q, W^V \in \mathbb{M}^{d_{emb} \times d_k}$ are **trainable parameters**

		...	
		...	
⋮	⋮	⋮	

$W^K (768 \times 64)$

		...	
		...	
⋮	⋮	⋮	

$W^Q (768 \times 64)$

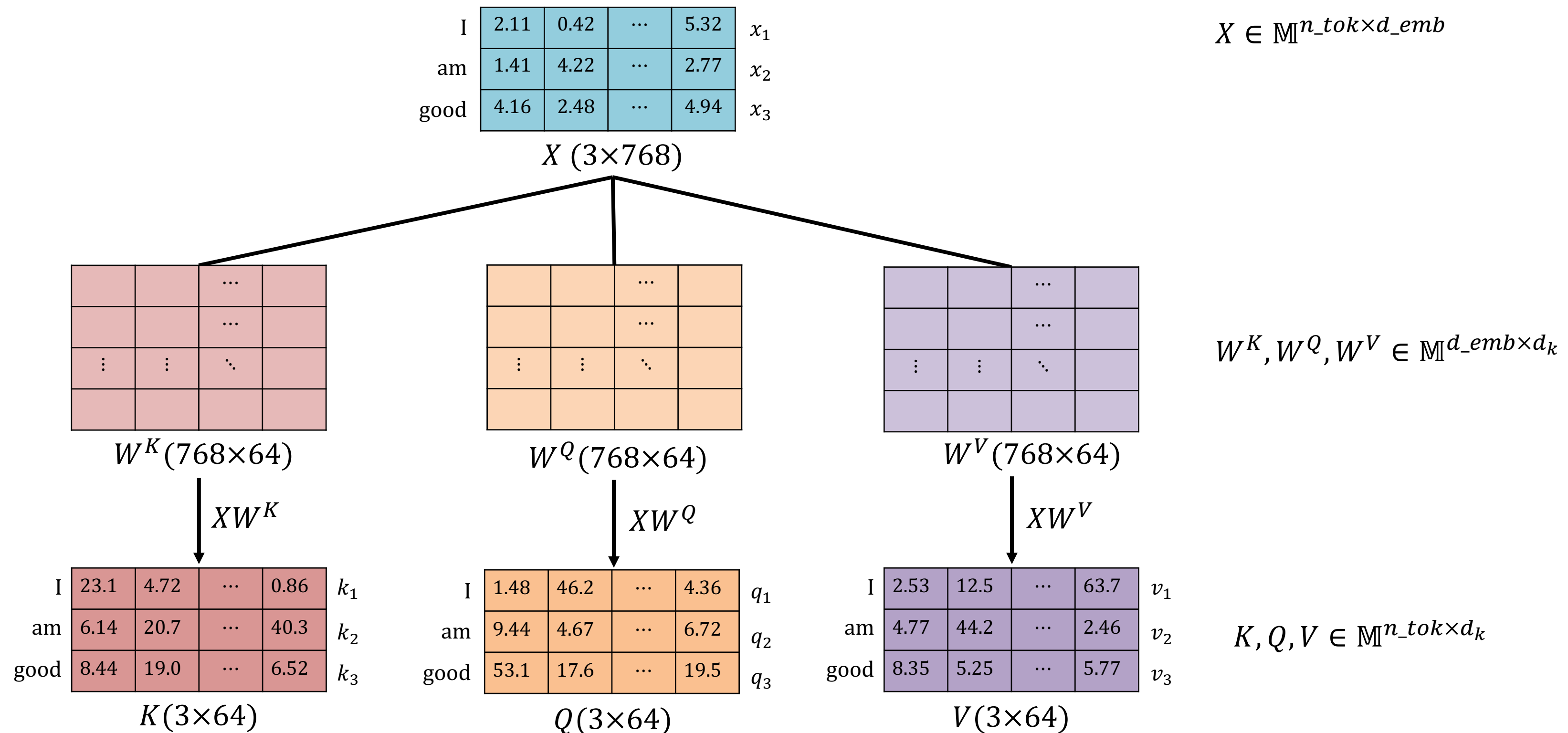
		...	
		...	
⋮	⋮	⋮	

$W^V (768 \times 64)$

Embedding size = 768, $d_k = 64$

01 Self-Attention (A real-word example)

- Generate K , Q and V by multiplying the weight matrices W^K, W^Q, W^V and the embedding matrix X



01 Self-Attention (A real-word example)

- Generate K , Q and V by multiplying the weight matrices W^K, W^Q, W^V and the embedding matrix X

$$K = XW^K$$

$$Q = XW^Q$$

$$V = XW^V$$

```
class AttentionHead(nn.Module):  
    def __init__(self, d_emb, d_k):  
        super().__init__()  
        self.W_q = nn.Linear(d_emb, d_k)  
        self.W_k = nn.Linear(d_emb, d_k)  
        self.W_v = nn.Linear(d_emb, d_k)
```

The weight matrix can also be generated with `nn.Embedding` of size 768x64 and then the weight matrix and embedding matrix can be internalized with `np.dot`, but in the above case it is easier to implement with `nn.Linear`.

01 Self-Attention (A real-word example)

Self Attention – Process

1. Extract embeddings from a sentence: create an embeddings matrix
2. Create Key, Query, Value from Embedded Matrix
3. Calculate the similarity between Query and Key : $Q \cdot K^T$
4. Scaling and Softmax (normalization) $\rightarrow$ score matrix : $\text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right)$
5. Multiply by Value to obtain Attachment matrix Z

01 Self-Attention (A real-word example)

3. Calculate the similarity between Query and Key : $Q \cdot K^T$

- Performing inner operations on the query and key matrices
- Computing the inner product of the two matrices gives the similarity for each token

I	1.48	46.2	...	4.36
am	9.44	4.67	...	6.72
good	53.1	17.6	...	19.5

q_1
 q_2
 q_3

 $\times$

I	am	good
23.1	6.14	8.44
4.72	20.7	19.0
$\vdots$	$\vdots$	$\vdots$
0.86	40.3	6.52

k_1 k_2 k_3
 $K^T(64 \times 3)$

 $=$

I	am	good
$q_1 \cdot k_1$	$q_1 \cdot k_2$	$q_1 \cdot k_3$
$q_2 \cdot k_1$	$q_2 \cdot k_2$	$q_2 \cdot k_3$
$q_3 \cdot k_1$	$q_3 \cdot k_2$	$q_3 \cdot k_3$

$QK^T(3 \times 3)$

01 Self-Attention (A real-word example)

3. Calculate the similarity between Query and Key : $Q \cdot K^T$

I	1.48	46.2	...	4.36
am	9.44	4.67	...	6.72
good	53.1	17.6	...	19.5

$\times$

I	23.1	6.14	8.44
am	4.72	20.7	19.0
good	0.86	40.3	6.52

$=$

I	$q_1 \cdot k_1$	$q_1 \cdot k_2$	$q_1 \cdot k_3$
am	$q_2 \cdot k_1$	$q_2 \cdot k_2$	$q_2 \cdot k_3$
good	$q_3 \cdot k_1$	$q_3 \cdot k_2$	$q_3 \cdot k_3$

$Q(3 \times 64)$

$K^T(64 \times 3)$

$QK^T(3 \times 3)$

- For the first row

Query : I , Key : I , am , good

→ Compute similarity to all tokens in a sentence for a query

01 Self-Attention (A real-word example)

3. Calculate the similarity between Query and Key : $Q \cdot K^T$

I	1.48	46.2	...	4.36
am	9.44	4.67	...	6.72
good	53.1	17.6	...	19.5

$\times$

I	23.1	6.14	8.44
am	4.72	20.7	19.0
good	0.86	40.3	6.52

$=$

I	$q_1 \cdot k_1$	$q_1 \cdot k_2$	$q_1 \cdot k_3$
am	$q_2 \cdot k_1$	$q_2 \cdot k_2$	$q_2 \cdot k_3$
good	$q_3 \cdot k_1$	$q_3 \cdot k_2$	$q_3 \cdot k_3$

$Q(3 \times 64)$

$K^T(64 \times 3)$

$QK^T(3 \times 3)$

- For the first row

Query : I , Key : I , am , good

→ Compute similarity to all tokens in a sentence for a query “I”

01 Self-Attention (A real-word example)

3. Calculate the similarity between Query and Key : $Q \cdot K^T$

I	1.48	46.2	...	4.36
am	9.44	4.67	...	6.72
good	53.1	17.6	...	19.5

 $Q(3 \times 64)$
 $\times$

I	am	good
23.1	6.14	8.44
4.72	20.7	19.0
$\vdots$	$\vdots$	$\vdots$
0.86	40.3	6.52

 $K^T(64 \times 3)$
 $=$

I	$q_1 \cdot k_1$	$q_1 \cdot k_2$	$q_1 \cdot k_3$
am	$q_2 \cdot k_1$	$q_2 \cdot k_2$	$q_2 \cdot k_3$
good	$q_3 \cdot k_1$	$q_3 \cdot k_2$	$q_3 \cdot k_3$

 $QK^T(3 \times 3)$

- For the first row

Query : am , Key : I , am , good

→ Compute similarity to all tokens in a sentence for a query “am”

01 Self-Attention (A real-word example)

3. Calculate the similarity between Query and Key : $Q \cdot K^T$

I	1.48	46.2	...	4.36
am	9.44	4.67	...	6.72
good	53.1	17.6	...	19.5

$Q(3 \times 64)$

$\times$

I	23.1	6.14	8.44
am	4.72	20.7	19.0
⋮	⋮	⋮	⋮
good	0.86	40.3	6.52

$K^T(64 \times 3)$

$=$

I	$q_1 \cdot k_1$	$q_1 \cdot k_2$	$q_1 \cdot k_3$
am	$q_2 \cdot k_1$	$q_2 \cdot k_2$	$q_2 \cdot k_3$
good	$q_3 \cdot k_1$	$q_3 \cdot k_2$	$q_3 \cdot k_3$

$QK^T(3 \times 3)$

- For the first row

Query : good , Key : I , am , good

→ Compute similarity to all tokens in a sentence for a query “good”

01 Self-Attention (A real-word example)

Self Attention – Process

1. Extract embeddings from a sentence: create an embeddings matrix
2. Create Key, Query, Value from Embedded Matrix
3. Calculate the similarity between Query and Key : $Q \cdot K^T$
4. Scaling and Softmax (normalization) → score matrix : $\text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right)$
5. Multiply by Value to obtain Attachment matrix Z

01 Self-Attention (A real-word example)

4. Scaling and Softmax (normalization) → score matrix : $\text{softmax} \left(\frac{Q \cdot K^T}{\sqrt{d_k}} \right)$

- Scaling: divide QK^T by the dimension of the key (scaled-dot product)
 - Scaling values that have become too large due to dot-product
 - Optimized gradient to be stable when back-propagating
- Softmax : Take the soft-max in $\frac{QK^T}{\sqrt{d_k}}$ and change it to a score matrix

01 Self-Attention (A real-word example)

Self Attention – Process

1. Extract embeddings from a sentence: create an embeddings matrix
2. Create Key, Query, Value from Embedded Matrix
3. Calculate the similarity between Query and Key : $Q \cdot K^T$
4. Scaling and Softmax (normalization) $\rightarrow$ score matrix : $\text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right)$
5. Multiply by Value to obtain Attachment matrix Z

01 Self-Attention (A real-word example)

5. Multiply by Value to obtain Attachment matrix Z

- Use the score matrix as a weight to give more weight to highly relevant words
 - Multiply the Score matrix and the Value matrix to get the Attention matrix Z .
- The model learns which words to focus on in which contexts through an attentional matrix.

	I	am	good
I	0.7	0.05	0.25
am	0.1	0.75	0.15
good	0.05	0.075	0.875

 $\times$

I	2.53	12.5	...	63.7	v_1
am	4.77	44.2	...	2.46	v_2
good	8.35	5.25	...	5.77	v_3

$softmax\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right) (3 \times 3)$
 $V(3 \times 64)$

01 Self-Attention (A real-word example)

Meaning of the Attention Matrix Z

Attention matrix Z has size 3x64 $Z = \begin{bmatrix} z_1 \\ z_2 \\ z_3 \end{bmatrix}$

$$z_1 = 0.90 \begin{bmatrix} 67.85 & 91.2 & \dots \end{bmatrix} + 0.07 \begin{bmatrix} 13.13 & 63.1 & \dots \end{bmatrix} + 0.03 \begin{bmatrix} 12.12 & 96.1 & \dots \end{bmatrix}$$

$v_1(\text{l}) \qquad \qquad \qquad v_2(\text{am}) \qquad \qquad \qquad v_3(\text{good})$

- z_1 is consisted of summation between 90% values of $v_1(\text{l})$, 7% values of $v_2(\text{am})$, and 3% values of $v_3(\text{good})$.
== The word “l” is 90% associated with “l”, 7% with “am”, and 3% with “good”.

01 Self-Attention (A real-word example)

- The operations in steps 3, 4, and 5 can be summarized as follows
- The input values are Q, K, and V from step 2.

```
def scaled_dot_product_attention(query, key, value):  
    dim_k = query.size(-1)  
    scores = torch.bmm(query, key.transpose(1, 2)) / sqrt(dim_k)  
    weights = F.softmax(scores, dim=-1)  
    return torch.bmm(weights, value)
```

✓ 0.0s

```
self_attention = scaled_dot_product_attention(q, k, v)  
self_attention.size()
```

✓ 0.0s

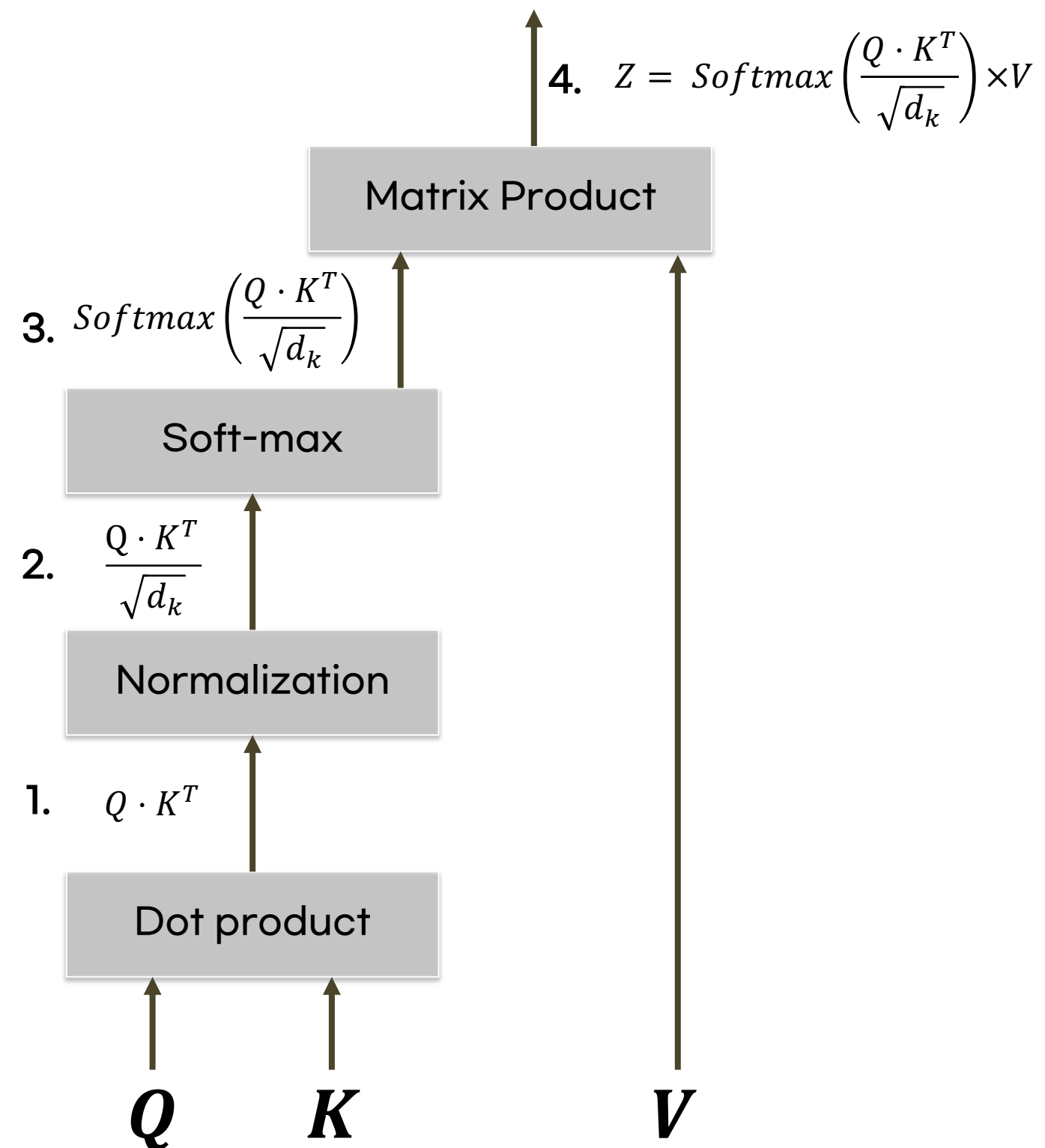
torch.Size([1, 7, 64])

※ torch.bmm is a batch matrix multiplication, used when both operators are batch.

$[B, N, M] \times [B, M, P] = [B, N, P]$

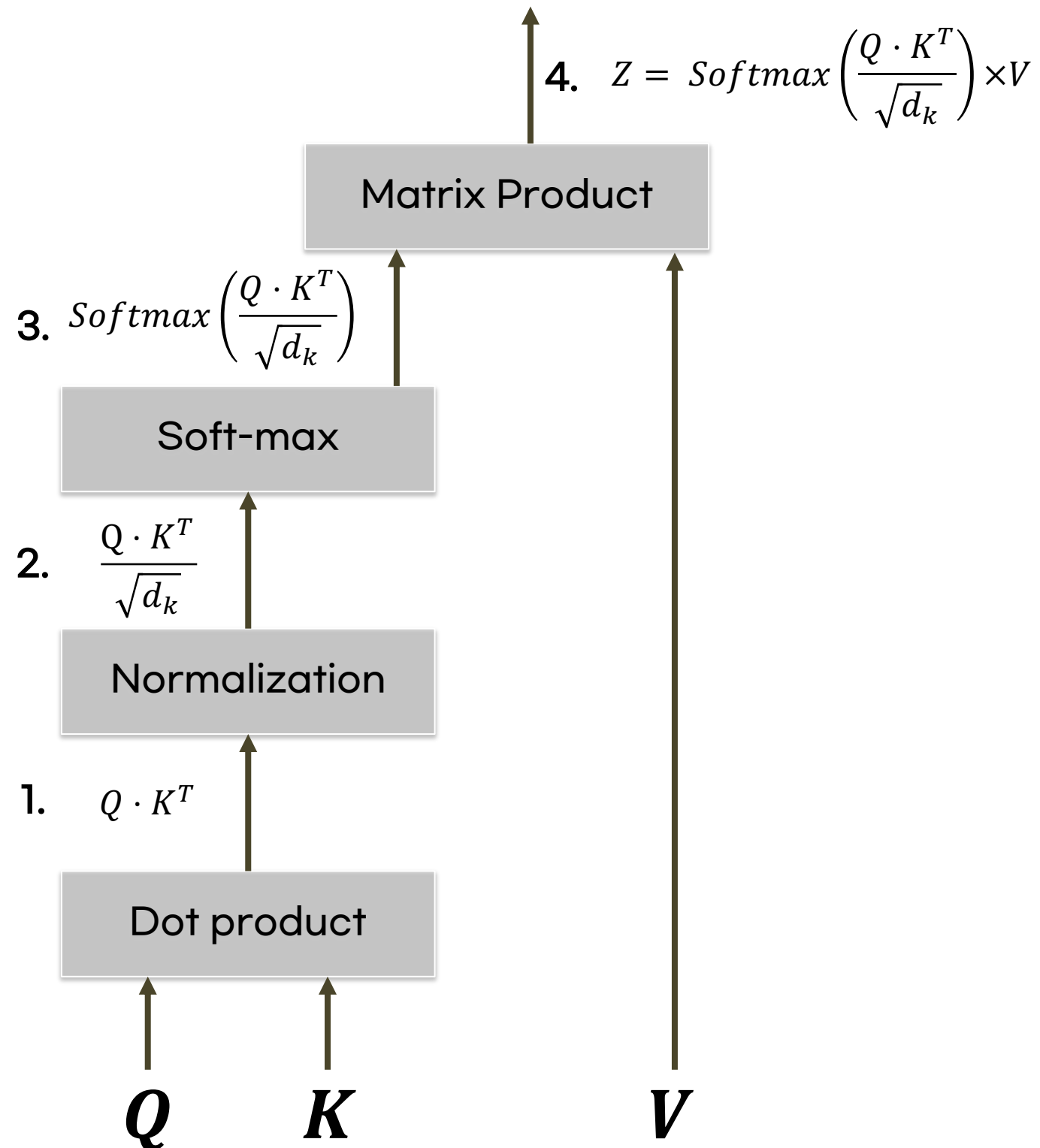
01 Self-Attention (A real-word example)

Self-Attention Summary



01 Self-Attention (A real-word example)

Self-Attention Summary



```
def scaled_dot_product_attention(query, key, value):  
    dim_k = query.size(-1)  
    scores = torch.bmm(query, key.transpose(1, 2))/sqrt(dim_k)  
    weights = F.softmax(scores, dim=-1)  
    return torch.bmm(weights, value)
```

```
class AttentionHead(nn.Module):  
    def __init__(self, X_embed_dim, d_k):  
        super().__init__()  
        self.W_q = nn.Linear(X_embed_dim, d_k)  
        self.W_k = nn.Linear(X_embed_dim, d_k)  
        self.W_v = nn.Linear(X_embed_dim, d_k)  
  
    def forward(self, sequence):  
        attn_output = scaled_dot_product_attention(  
            self.W_q(sequence), self.W_k(sequence), self.W_v(sequence)  
        )
```


Contents

01 Introduction of Transformer

1) Encoder-Decoder Structure

2) Transformer Structure

2) Usage of Encoder-Decoder

02 Transformer Encoder

1) Self-Attention

2) Multi-Head Attention

3) Positional Encoding

4) Add & Norm



Contents

01 Introduction of Transformer

1) Encoder-Decoder Structure

2) Transformer Structure

2) Usage of Encoder-Decoder

02 Transformer Encoder

1) Self-Attention

2) Multi-Head Attention

3) Positional Encoding

4) Add & Norm



02 Multi-Head Attention

- Multi-Head Attention means using multiple Self-Attention to set the Attention matrix

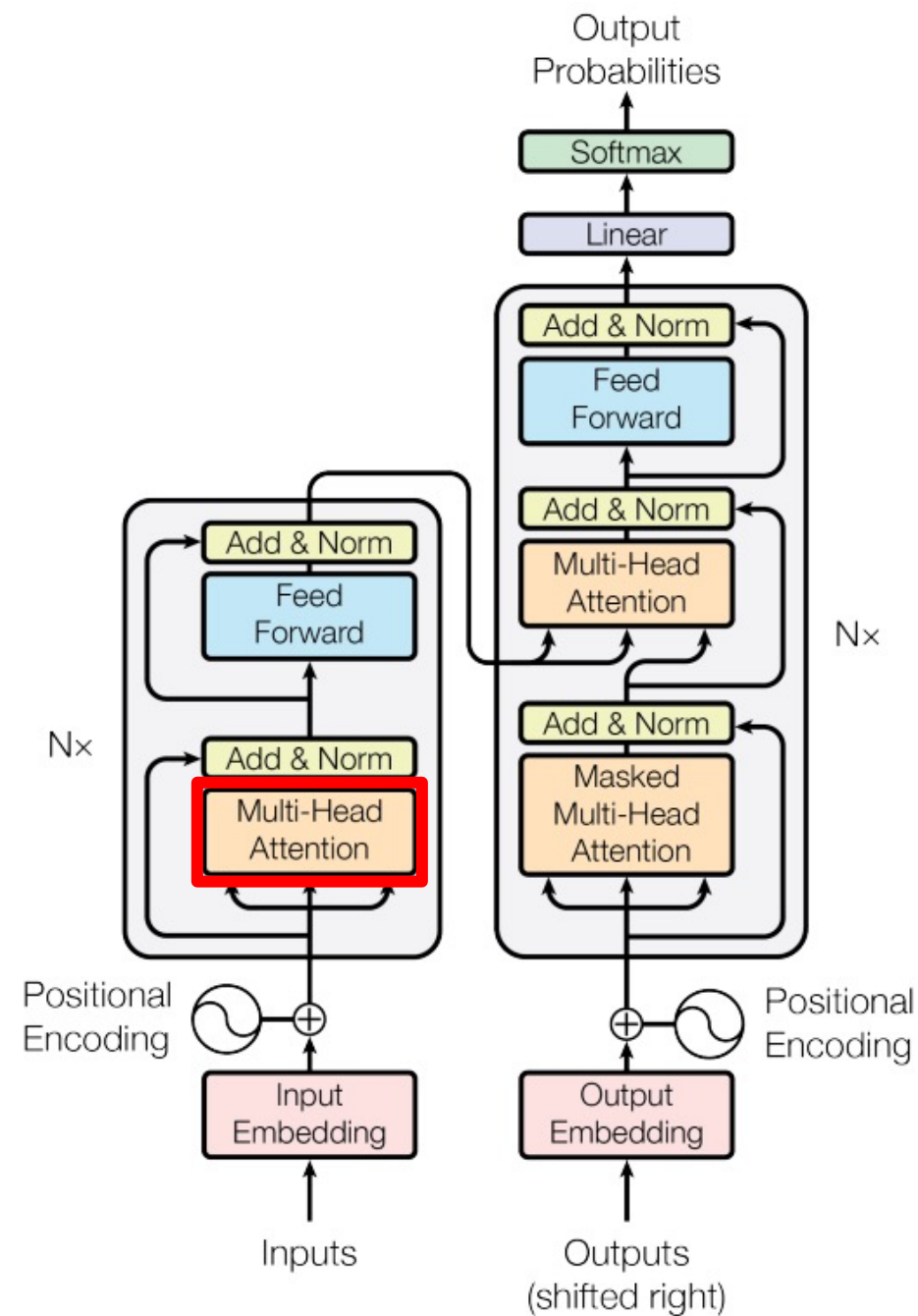
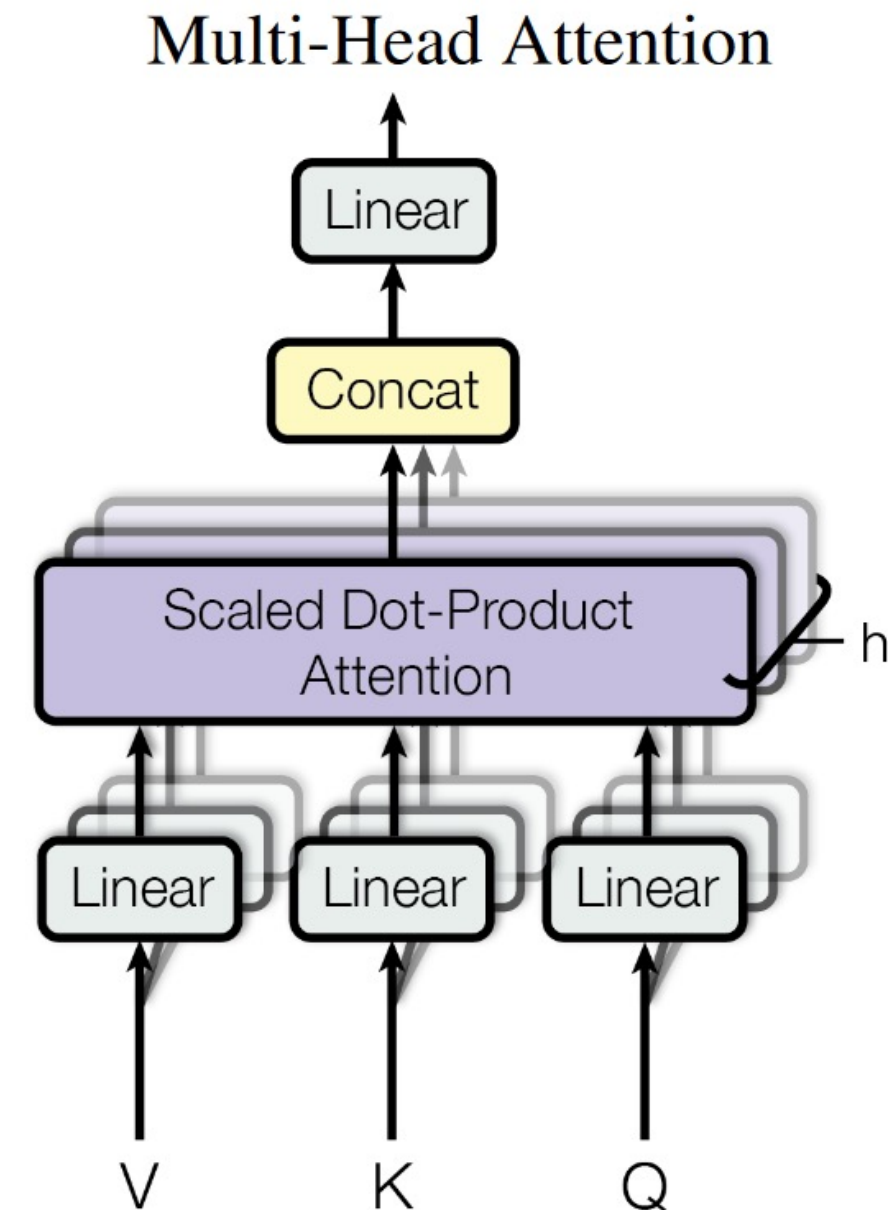
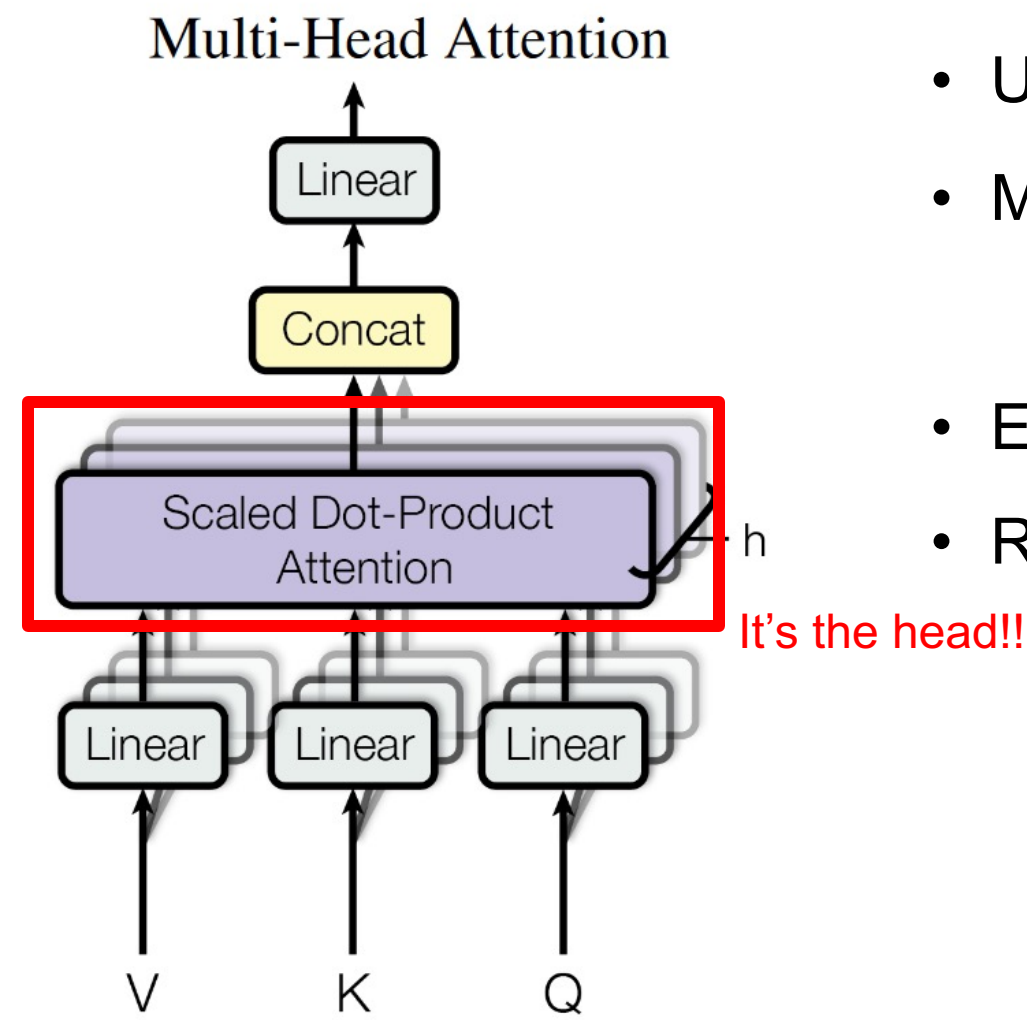


Figure 1: The Transformer - model architecture.



02 Multi-Head Attention

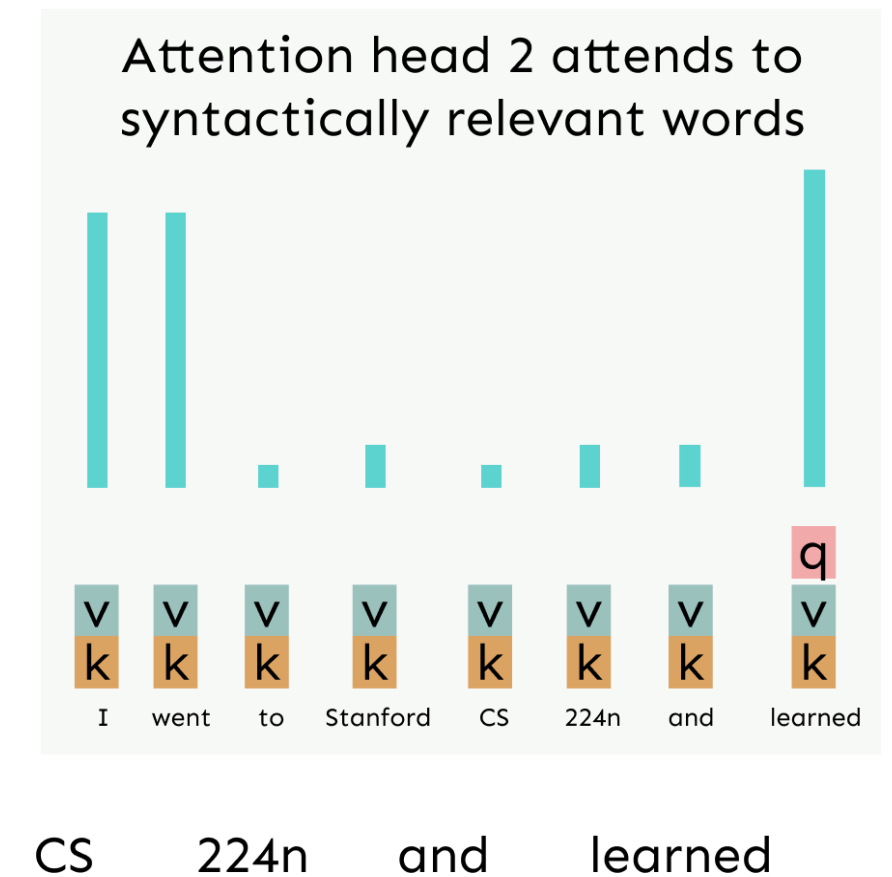
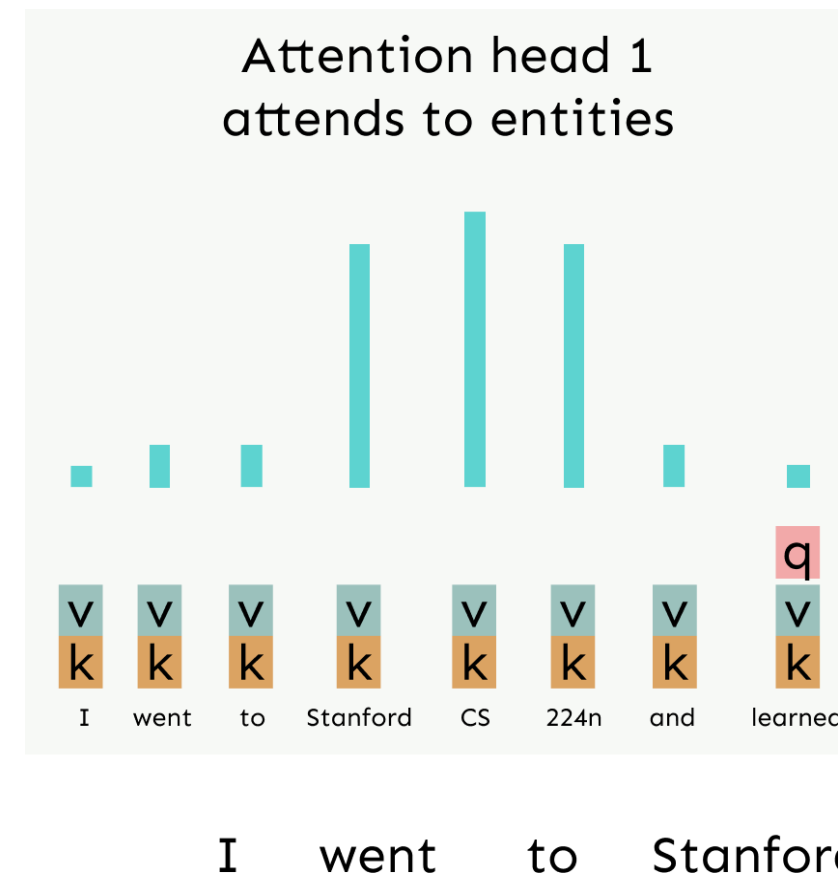
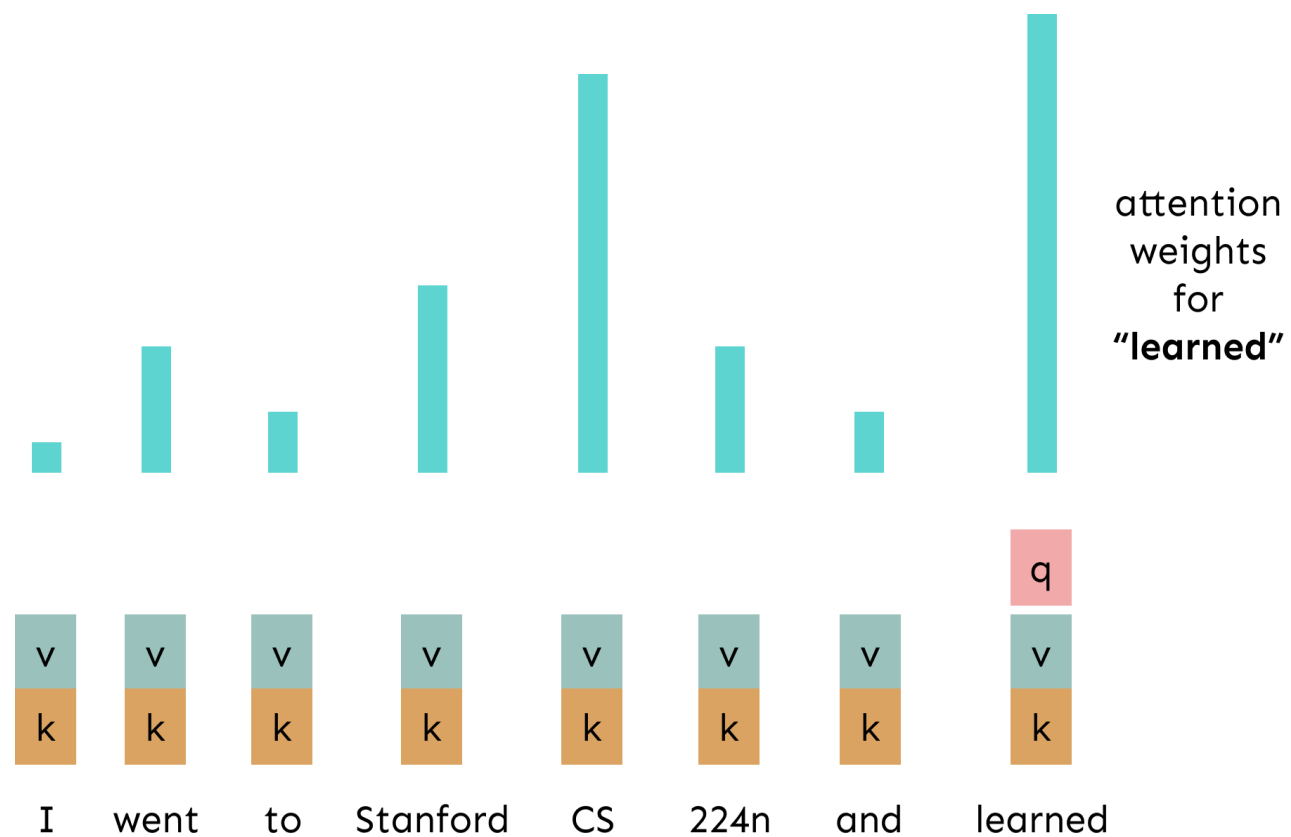
- Multi-Head Attention means using multiple Self-Attention to set the Attention matrix



- Used to increase the accuracy of Attention results
- Mitigating the tendency to focus on specific tokens when using a single Self-Attention.
 - Like an ensemble
- Effective for GPU computation by performing Self-Attention in parallel
- Reducing the embedding size by reducing the computation due to more heads

02 Multi-Head Attention

- Multi-Head Attention means using multiple Self-Attention to set the Attention matrix



02 Multi-Head Attention

- Assuming we have two heads of Multi-Head Attention, the calculation is as follows

1. Compute attention heads, Z_1, Z_2

for head_no in range(num_head):

1) Set W_1^Q, W_1^K, W_1^V as we have done and compute Q_1, K_1, V_1 . Then $Z_1 = \text{softmax}\left(\frac{Q_1 K_1^T}{\sqrt{d_k}}\right) V_1$

2) Set W_2^Q, W_2^K, W_2^V as we have done and compute Q_2, K_2, V_2 . Then $Z_2 = \text{softmax}\left(\frac{Q_2 K_2^T}{\sqrt{d_k}}\right) V_2$

2. Concatenate all heads and then do a linear transformation

$$\text{concatenate}([Z_1, Z_2])W$$

02 Multi-Head Attention

- Concatenate all heads and then do a linear transformation

$$\text{concatenate}([Z_1, Z_2])W$$

Why we need a linear projection?



Z_1 and Z_2 are obtained through different weights.

→ Z_1 and Z_2 are in different subspaces

→ Vectors in different subspaces need to be projected onto the same space

→ Mapping through Linear Layer!!

02 Multi-Head Attention

- Why we need a linear projection?

When the number of heads is h , we have h number of single-head attention $Z_1, \dots, Z_h$ simply concatenate it

$$\text{Concatenate}([Z_1, Z_2, \dots, Z_h])W$$

03 Positional Encoding

- Transformer takes in tokens in parallel, so the order of the tokens is not preserved
→ To solve this, use positional encoding!

In the original paper, the position encoding was computed via a sine wave function, which was then added to the embedding matrix and position encoding.

In this work, we use sine and cosine functions of different frequencies:

$$PE_{(pos,2i)} = \sin(pos/10000^{2i/d_{model}})$$

$$PE_{(pos,2i+1)} = \cos(pos/10000^{2i/d_{model}})$$

$$P = \begin{bmatrix} \text{I} & \sin(0) & \cos(0) & \sin(0/100) & \cos(0/100) \\ \text{am} & \sin(1) & \cos(1) & \sin(1/100) & \cos(1/100) \\ \text{good} & \sin(2) & \cos(2) & \sin(2/100) & \cos(2/100) \end{bmatrix}$$

The size of the
embedding matrix
... =
Size of the positional
encoding

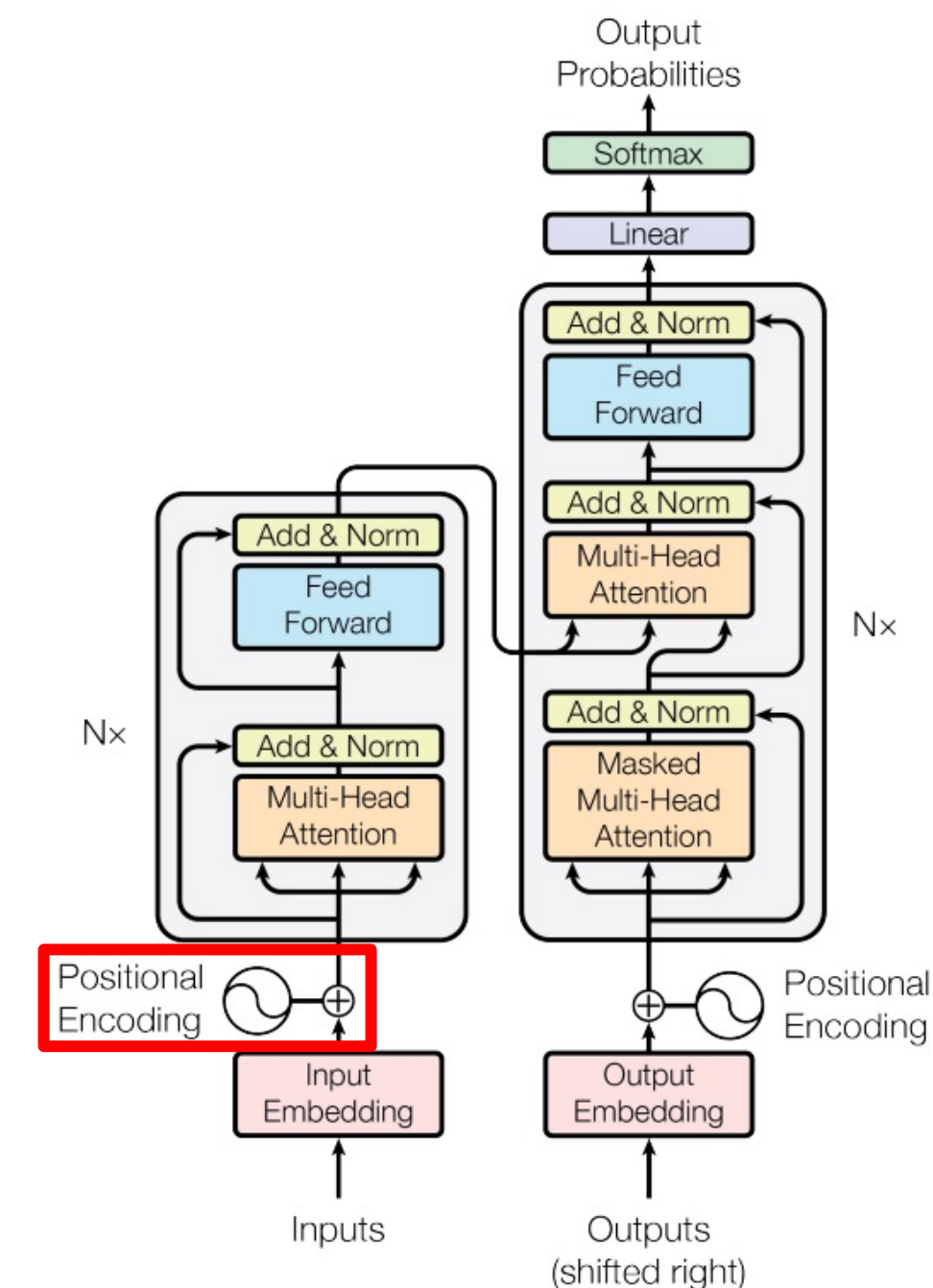


Figure 1: The Transformer - model architecture.

04 Add&Norm

- **Add:** means Skip-connection and acts as a normalizer so that the input has a mean of 0 and variance of 1 (prevents Vanishing Gradient)
- **Norm** : Layer Normalization - Sequence is divided into post-layer normalization, pre-layer normalization according to the position of LayerNorm respectively.

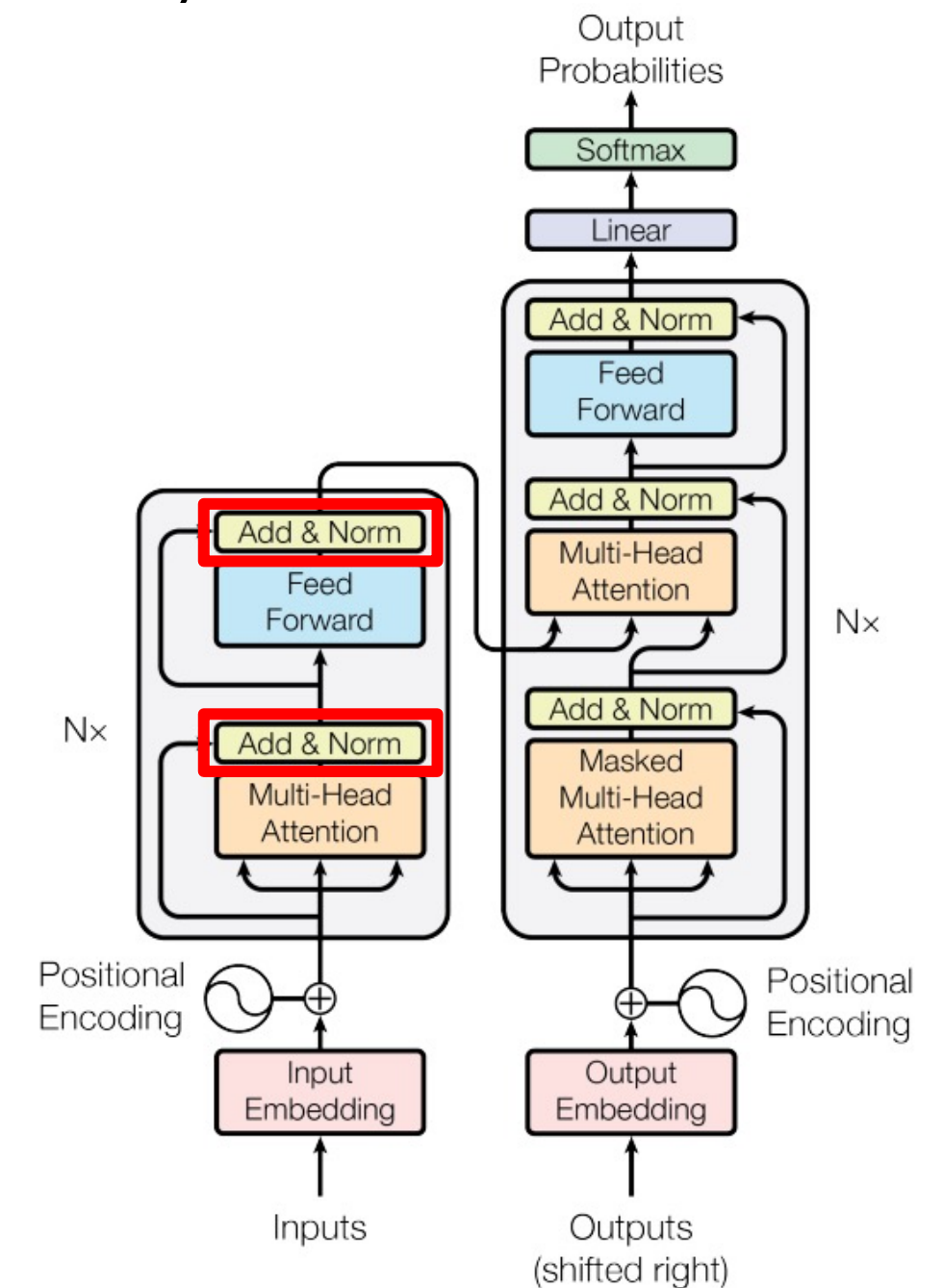


Figure 1: The Transformer - model architecture.

05 FFN

- A Feed Forward Network (FFN) typically sizes the first layer to be four times the embedding and uses GELU as the activation function.

```
class FeedForward(nn.Module):  
    def __init__(self, config):  
        super().__init__()  
        self.linear_1 = nn.Linear(config.hidden_size, config.intermediate_size)  
        self.linear_2 = nn.Linear(config.intermediate_size, config.hidden_size)  
        self.gelu = nn.GELU()  
        self.dropout = nn.Dropout(config.hidden_dropout_prob)  
  
    def forward(self, x):  
        x = self.linear_1(x)  
        x = self.gelu(x)  
        x = self.linear_2(x)  
        x = self.dropout(x)  
        return x
```

✓ 0.0s

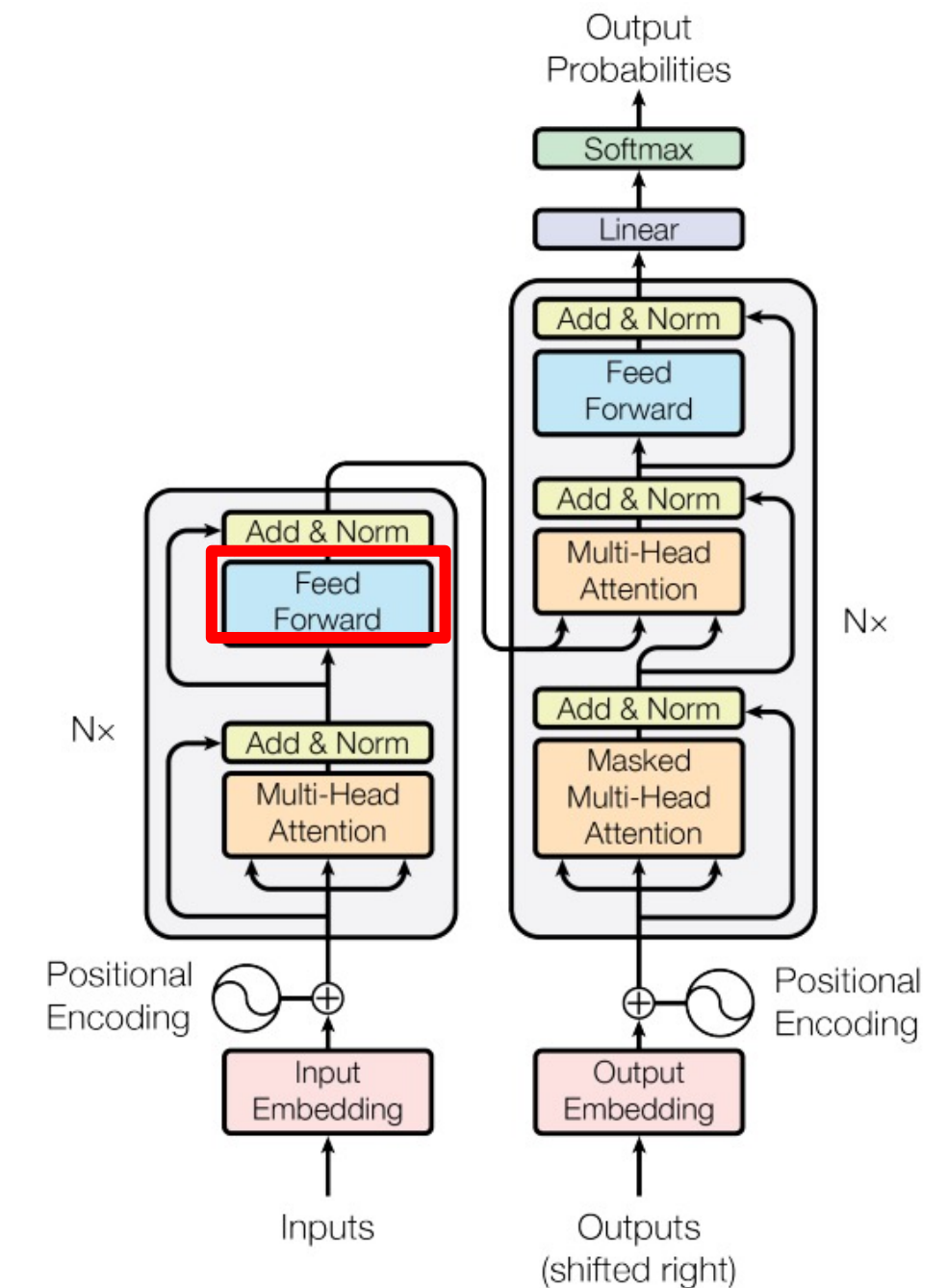


Figure 1: The Transformer - model architecture.

00 Transformer Encoder code

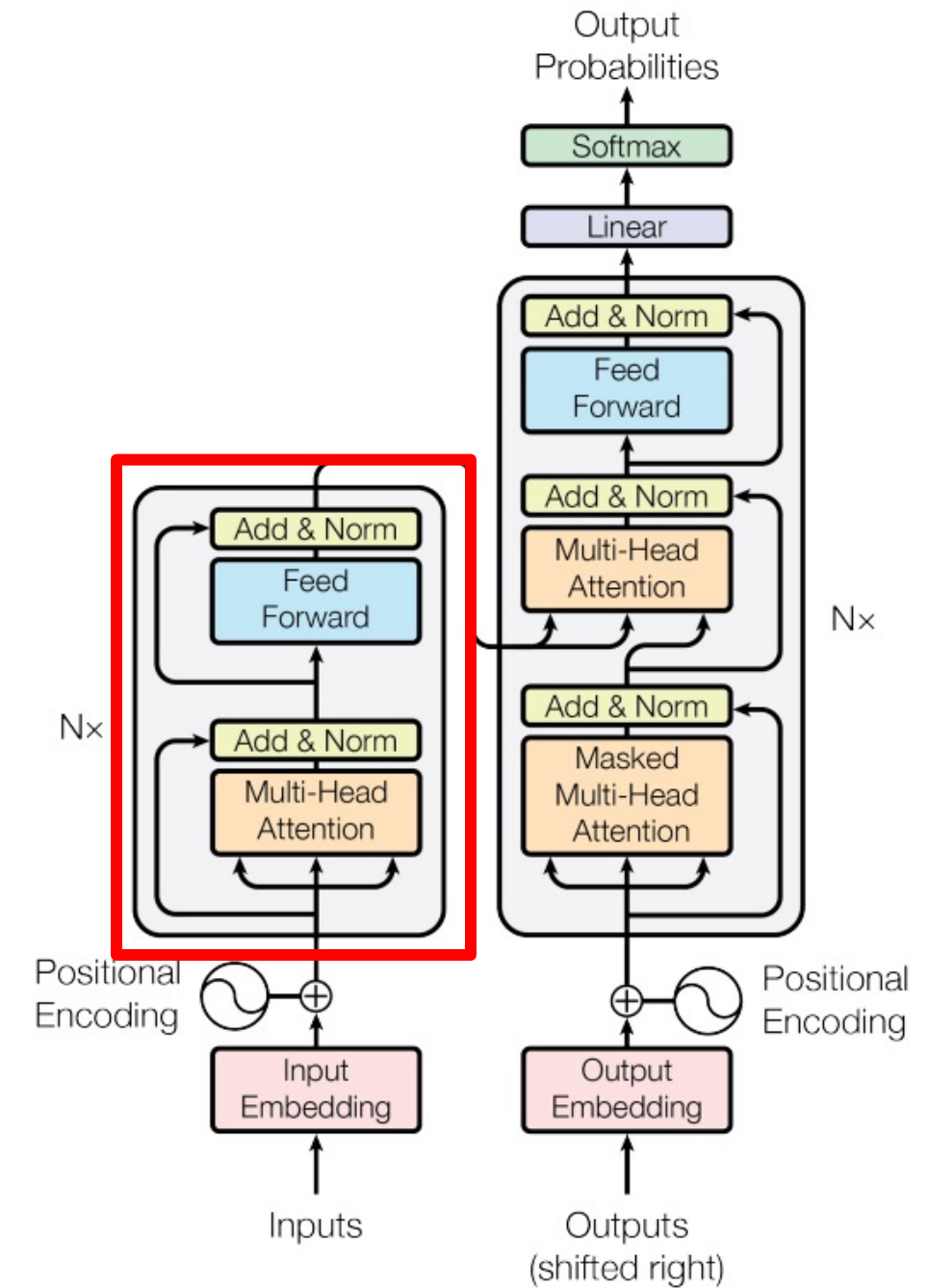
- Overall Code structure

```
class TransformerEncoderLayer(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.layer_norm_1 = nn.LayerNorm(config.hidden_size)
        self.layer_norm_2 = nn.LayerNorm(config.hidden_size)
        self.attention = MultiHeadAttention(config)
        self.feed_forward = FeedForward(config)

    def forward(self, x):
        attention_result = self.attention(x)
        x = x + attention_result
        ffn_result = self.feed_forward(x)
        x = self.layer_norm_2(x+ffn_result)
        return x
```

```
encoder_layer = TransformerEncoderLayer(config)
inputs_embeds.shape, encoder_layer(inputs_embeds).size()
```

```
(torch.Size([1, 7, 768]), torch.Size([1, 7, 768]))
```



00 Transformer Encoder code

```
from torch import nn
from transformers import AutoConfig
import torch.nn.functional as F
from transformers import AutoTokenizer
import torch
from math import sqrt
```

✓ 0.0s

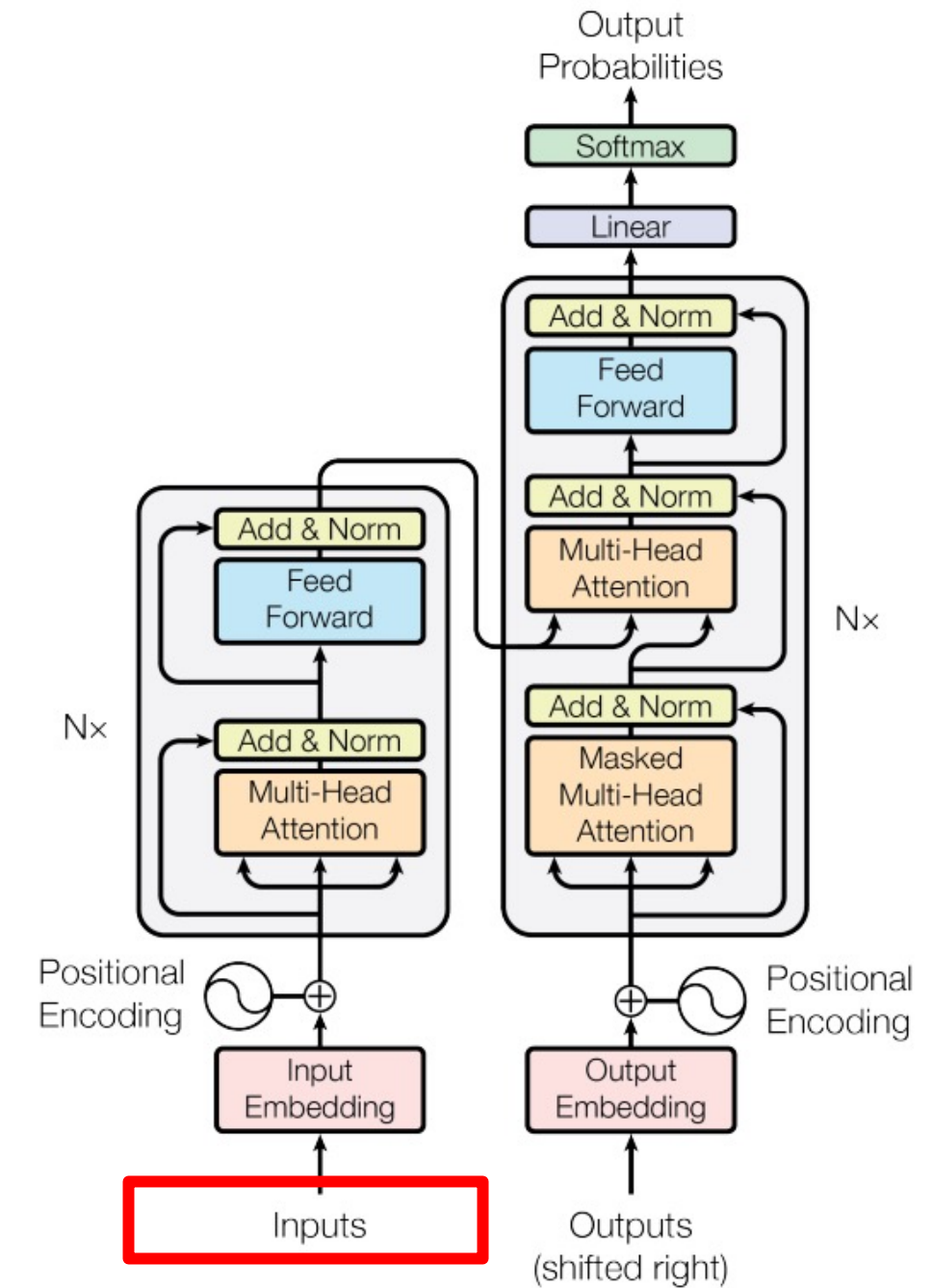
```
model_ckpt = "bert-base-uncased"
text = "time flies like an arrow"
tokenizer = AutoTokenizer.from_pretrained(model_ckpt)
config = AutoConfig.from_pretrained(model_ckpt)
```

✓ 2.7s

```
inputs = tokenizer(text, return_tensors="pt", add_special_tokens=True)
print(inputs.input_ids)
print(tokenizer.decode(inputs.input_ids[0]))
```

✓ 0.0s

```
tensor([[ 101, 2051, 10029, 2066, 2019, 8612, 102]])
[CLS] time flies like an arrow [SEP]
```



00 Transformer Encoder code

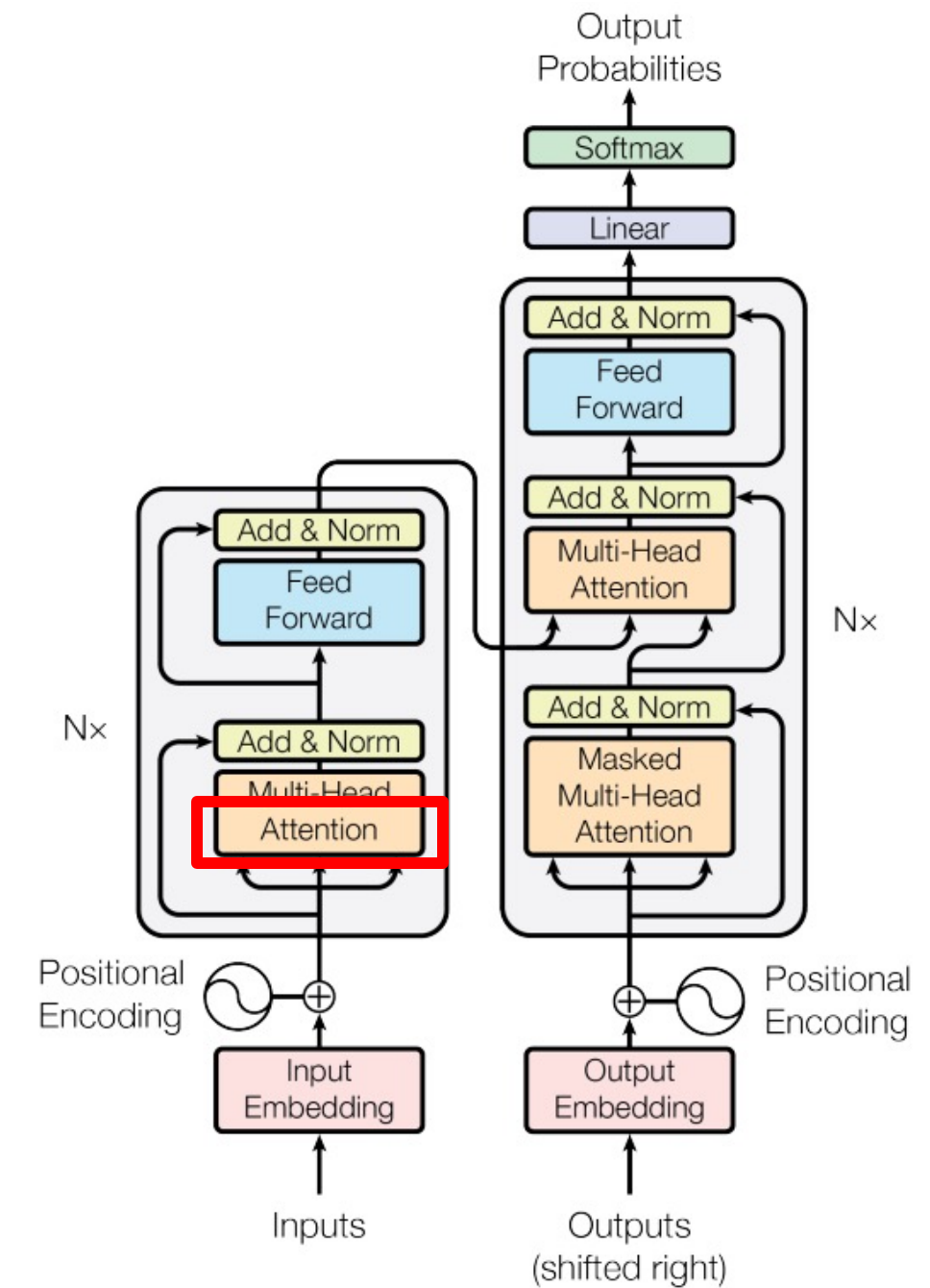
```
def scaled_dot_product_attention(query, key, value):
    dim_k = query.size(-1)
    scores = torch.bmm(query, key.transpose(1, 2)) / sqrt(dim_k)
    weights = F.softmax(scores, dim=-1)
    return torch.bmm(weights, value)
```

✓ 0.0s

```
class AttentionHead(nn.Module):
    def __init__(self, embed_dim, head_dim):
        super().__init__()
        self.W_q = nn.Linear(embed_dim, head_dim)
        self.W_k = nn.Linear(embed_dim, head_dim)
        self.W_v = nn.Linear(embed_dim, head_dim)

    def forward(self, hidden_state):
        attn_outputs = scaled_dot_product_attention(
            self.W_q(hidden_state),
            self.W_k(hidden_state),
            self.W_v(hidden_state)
        )
        return attn_outputs
```

✓ 0.0s

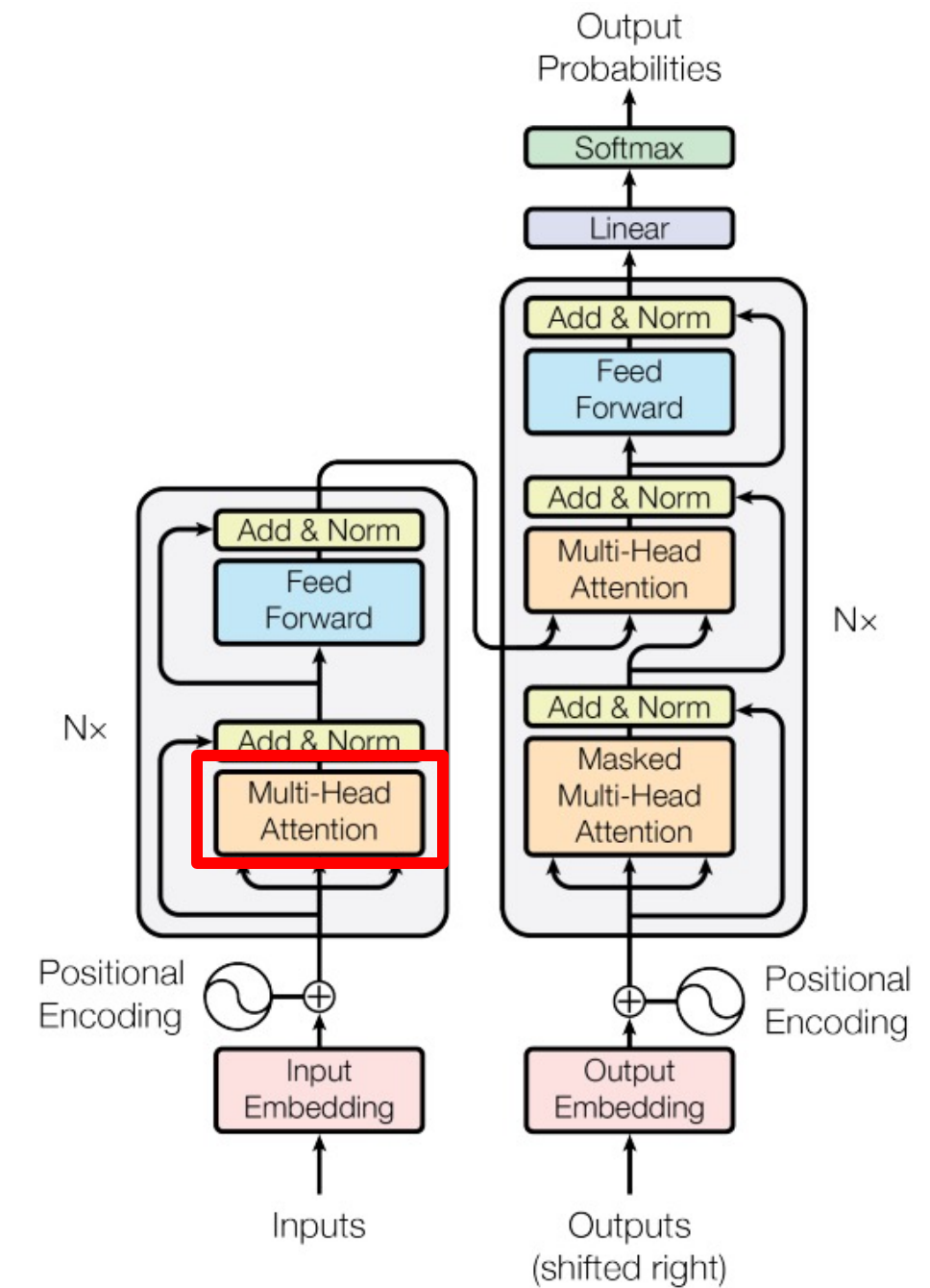


00 Transformer Encoder code

```
class MultiHeadAttention(nn.Module):
    def __init__(self, config):
        super().__init__()
        embed_dim = config.hidden_size
        num_heads = config.num_attention_heads
        head_dim = embed_dim // num_heads
        self.heads = nn.ModuleList(
            [AttentionHead(embed_dim, head_dim) for _ in range(num_heads)]
        )
        self.output_linear = nn.Linear(embed_dim, embed_dim)

    def forward(self, hidden_state):
        x = torch.cat([h(hidden_state) for h in self.heads], dim=-1)
        x = self.output_linear(x)
        return x
```

✓ 0.0s

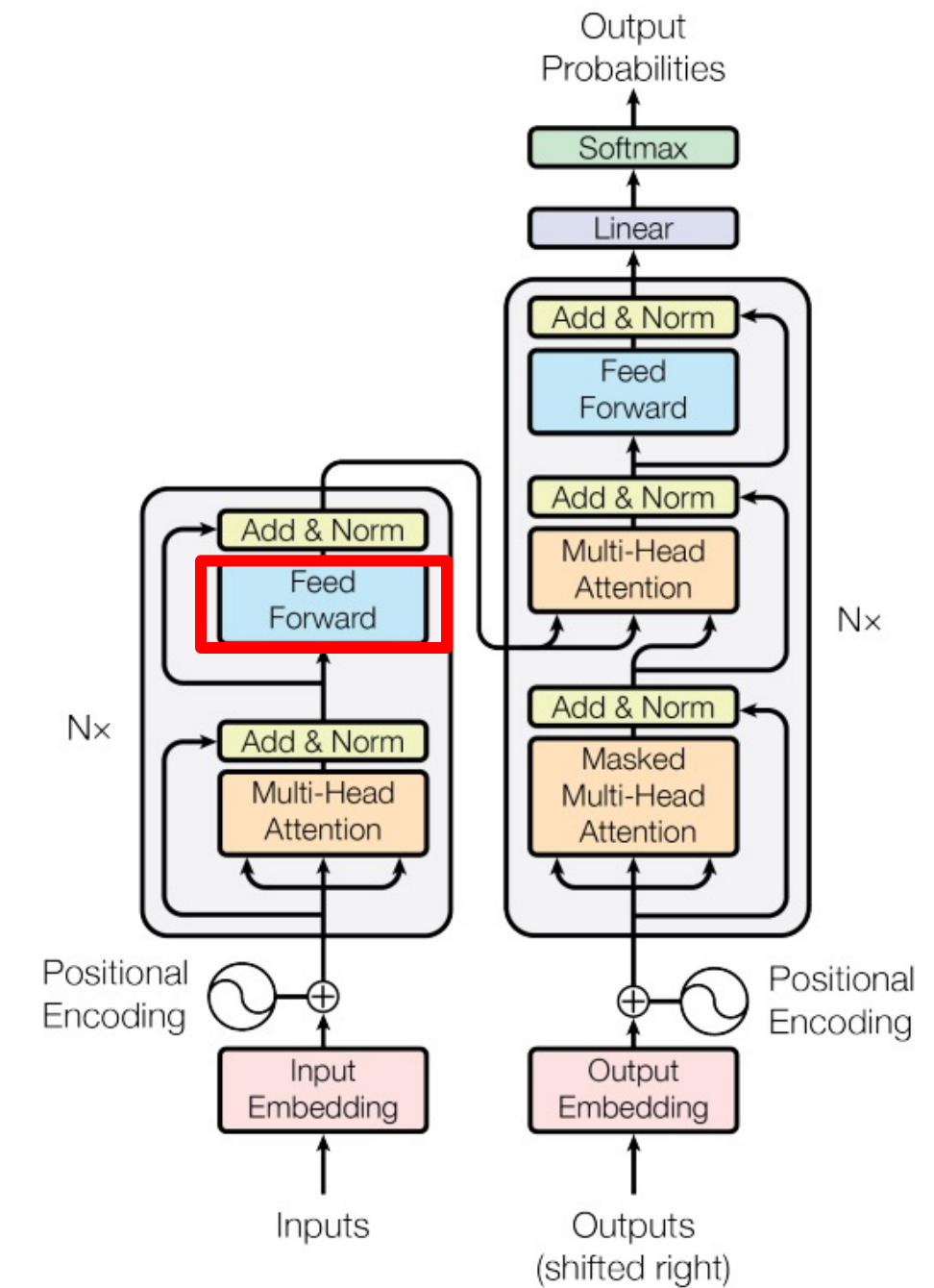


00 Transformer Encoder code

```
class FeedForward(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.linear_1 = nn.Linear(config.hidden_size, config.intermediate_size)
        self.linear_2 = nn.Linear(config.intermediate_size, config.hidden_size)
        self.gelu = nn.GELU()
        self.dropout = nn.Dropout(config.hidden_dropout_prob)

    def forward(self, x):
        x = self.linear_1(x)
        x = self.gelu(x)
        x = self.linear_2(x)
        x = self.dropout(x)
        return x
```

✓ 0.0s

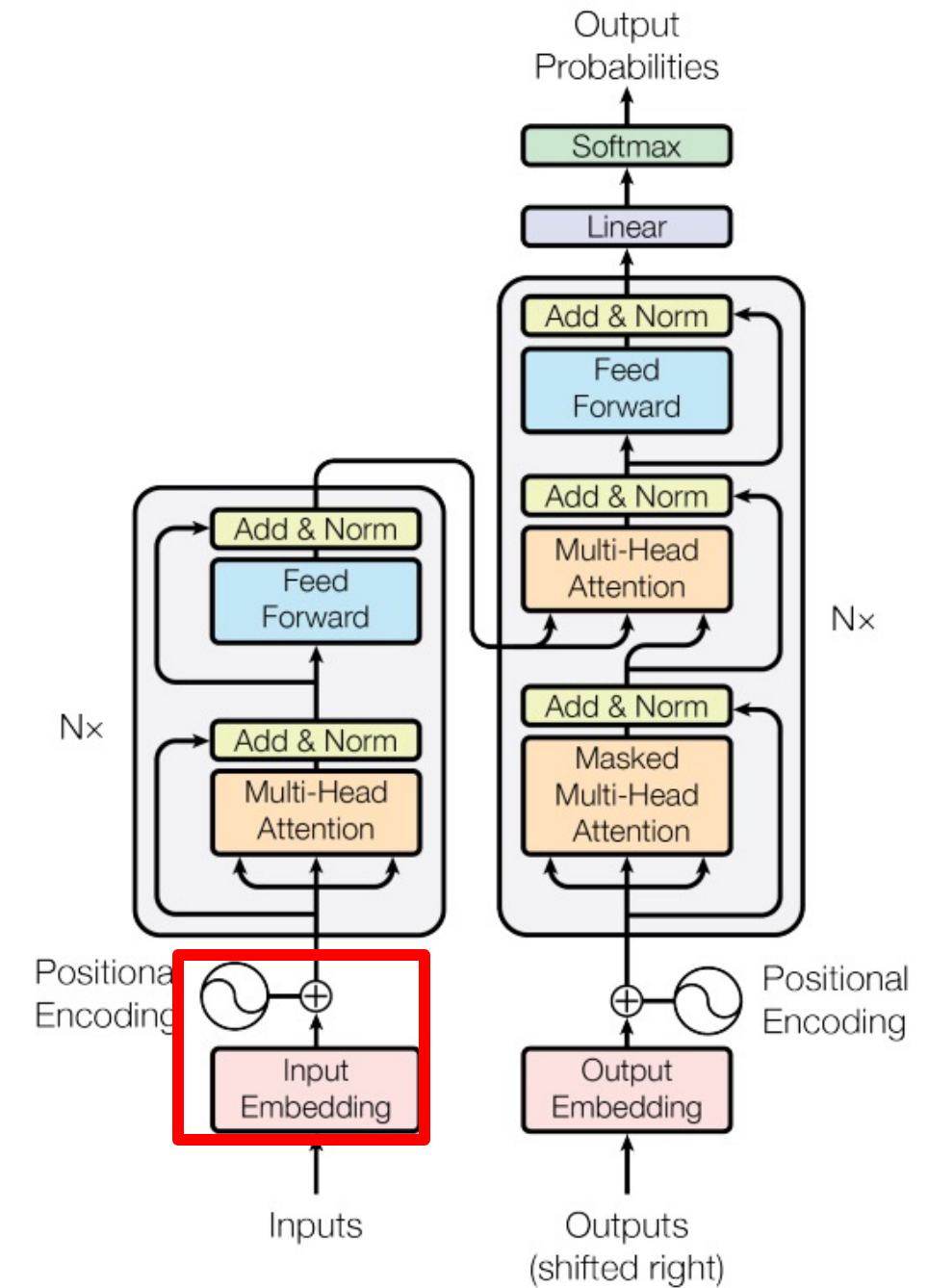


00 Transformer Encoder code

```
#토큰 임베딩과 위치 임베딩을 동시에
class Embeddings(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.token_embeddings = nn.Embedding(config.vocab_size,
                                             config.hidden_size)
        self.position_embeddings = nn.Embedding(config.max_position_embeddings,
                                                config.hidden_size)
        self.layer_norm = nn.LayerNorm(config.hidden_size, eps=1e-12)
        self.dropout = nn.Dropout()

    def forward(self, input_ids):
        # 위치 id 생성
        seq_length = input_ids.size(1)
        position_ids = torch.arange(seq_length, dtype=torch.long).unsqueeze(0)
        # 토큰 임베딩, 위치 임베딩 생성
        token_embeddings = self.token_embeddings(input_ids)
        position_embeddings = self.position_embeddings(position_ids)
        # 토큰 임베딩, 위치 임베딩 결합
        embeddings = token_embeddings + position_embeddings
        embeddings = self.layer_norm(embeddings)
        embeddings = self.dropout(embeddings)
        return embeddings
```

✓ 0.0s



00 Transformer Encoder code

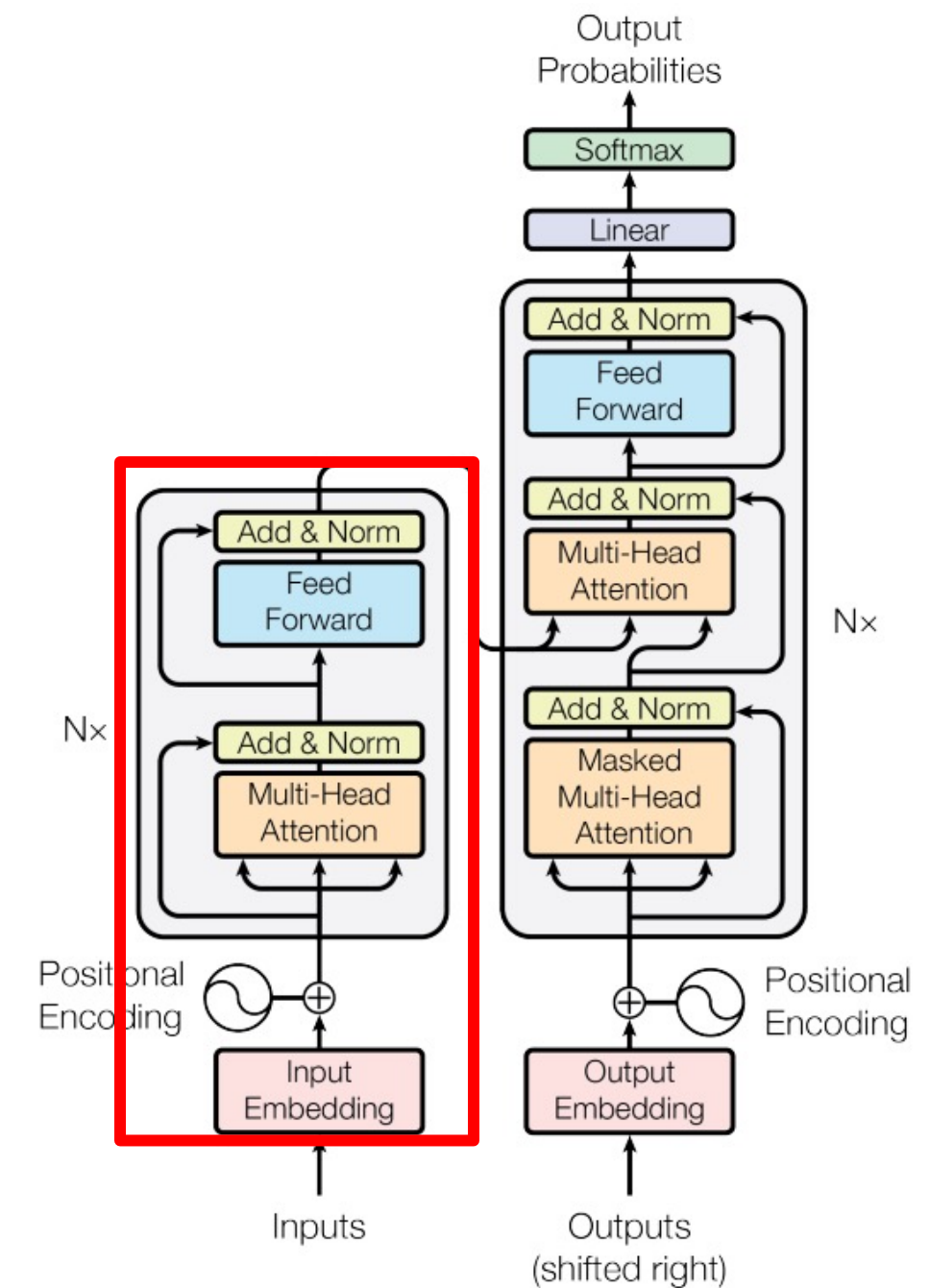
```
class TransformerEncoder(nn.Module):  
    def __init__(self, config):  
        super().__init__()  
        self.embeddings = Embeddings(config)  
        self.layers = nn.ModuleList([TransformerEncoderLayer(config)  
                                     for _ in range(config.num_hidden_layers)])  
  
    def forward(self, x):  
        x = self.embeddings(x)  
        for layer in self.layers:  
            x = layer(x)  
        return x
```

✓ 0.0s

```
encoder = TransformerEncoder(config)  
encoder(inputs.input_ids).size()
```

✓ 0.5s

torch.Size([1, 7, 768])



- The end result is a 768-dimensional Representation for 7 tokens.

Thank you